Overview

Overview

Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

This page is an overview of Kubernetes.

The name Kubernetes originates from Greek, meaning helmsman or pilot. K8s as an abbreviation results from counting the eight letters between the "K" and the "s". Google open sourced the Kubernetes project in 2014. Kubernetes combines over 15 years of Google's experience running production workloads at scale with best-of-breed ideas and practices from the community.

Why you need Kubernetes and what it can do

Containers are a good way to bundle and run your applications. In a production environment, you need to manage the containers that run the applications and ensure that there is no downtime. For example, if a container goes down, another container needs to start. Wouldn't it be easier if this behavior was handled by a system?

That's how Kubernetes comes to the rescue! Kubernetes provides you with a framework to run distributed systems resiliently. It takes care of scaling and failover for your application, provides deployment patterns, and more. For example: Kubernetes can easily manage a canary deployment for your system.

Kubernetes provides you with:

- **Service discovery and load balancing** Kubernetes can expose a container using a DNS name or its own IP address. If traffic to a container is high, Kubernetes is able to load balance and distribute the network traffic so that the deployment is stable.
- **Storage orchestration** Kubernetes allows you to automatically mount a storage system of your choice, such as local storage, public cloud providers, and more.
- **Automated rollouts and rollbacks** You can describe the desired state for your deployed containers using Kubernetes, and it can change the actual state to the desired state at a controlled rate. For example, you can automate Kubernetes to create new containers for your deployment, remove existing containers and adopt all their resources to the new container.
- **Automatic bin packing** You provide Kubernetes with a cluster of nodes that it can use to run containerized tasks. You tell Kubernetes how much CPU and memory (RAM) each container needs. Kubernetes can fit containers onto your nodes to make the best use of your resources.
- **Self-healing** Kubernetes restarts containers that fail, replaces containers, kills containers that don't respond to your user-defined health check, and doesn't advertise them to clients until they are ready to serve.
- **Secret and configuration management** Kubernetes lets you store and manage sensitive information, such as passwords, OAuth tokens, and SSH keys. You can deploy and update secrets and application configuration without rebuilding your container images, and without exposing secrets in your stack configuration.
- **Batch execution** In addition to services, Kubernetes can manage your batch and CI workloads, replacing containers that fail, if desired.
- **Horizontal scaling** Scale your application up and down with a simple command, with a UI, or automatically based on CPU usage.
- **IPv4/IPv6 dual-stack** Allocation of IPv4 and IPv6 addresses to Pods and Services
- **Designed for extensibility** Add features to your Kubernetes cluster without changing upstream source code.

What Kubernetes is not

Kubernetes is not a traditional, all-inclusive PaaS (Platform as a Service) system. Since Kubernetes operates at the container level rather than at the hardware level, it provides some generally applicable features common to PaaS offerings, such as deployment, scaling, load balancing, and lets users integrate their logging, monitoring, and alerting solutions. However, Kubernetes is not monolithic, and these

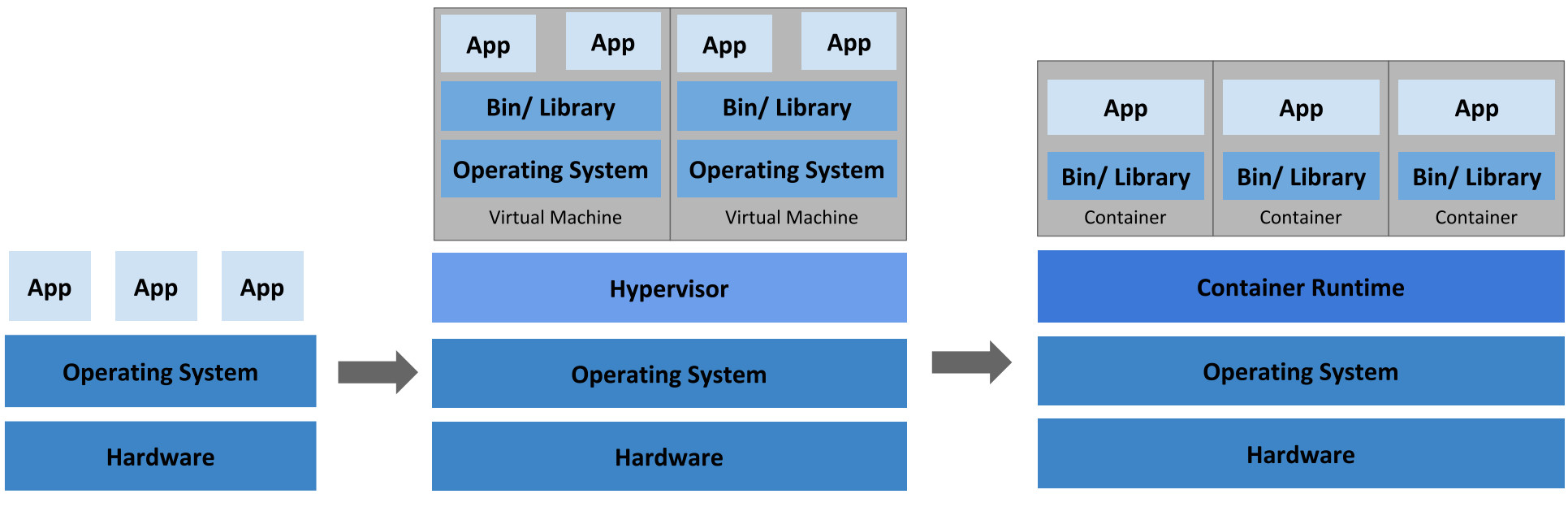
default solutions are optional and pluggable. Kubernetes provides the building blocks for building developer platforms, but preserves user choice and flexibility where it is important.

Kubernetes:

- Does not limit the types of applications supported. Kubernetes aims to support an extremely diverse variety of workloads, including stateless, stateful, and data-processing workloads. If an application can run in a container, it should run great on Kubernetes.
- Does not deploy source code and does not build your application. Continuous Integration, Delivery, and Deployment (CI/CD) workflows are determined by organization cultures and preferences as well as technical requirements.
- Does not provide application-level services, such as middleware (for example, message buses), data-processing frameworks (for example, Spark), databases (for example, MySQL), caches, nor cluster storage systems (for example, Ceph) as built-in services. Such components can run on Kubernetes, and/or can be accessed by applications running on Kubernetes through portable mechanisms, such as the Open Service Broker.
- Does not dictate logging, monitoring, or alerting solutions. It provides some integrations as proof of concept, and mechanisms to collect and export metrics.
- Does not provide nor mandate a configuration language/system (for example, Jsonnet). It provides a declarative API that may be targeted by arbitrary forms of declarative specifications.
- Does not provide nor adopt any comprehensive machine configuration, maintenance, management, or self-healing systems.
- Additionally, Kubernetes is not a mere orchestration system. In fact, it eliminates the need for orchestration. The technical definition of orchestration is execution of a defined workflow: first do A, then B, then C. In contrast, Kubernetes comprises a set of independent, composable control processes that continuously drive the current state towards the provided desired state. It shouldn't matter how you get from A to C. Centralized control is also not required. This results in a system that is easier to use and more powerful, robust, resilient, and extensible.

Historical context for Kubernetes

Let's take a look at why Kubernetes is so useful by going back in time.



Traditional deployment era:

Early on, organizations ran applications on physical servers. There was no way to define resource boundaries for applications in a physical server, and this caused resource allocation issues. For example, if multiple applications run on a physical server, there can be instances where one application would take up most of the resources, and as a result, the other applications would underperform. A solution for this would be to run each application on a different physical server. But this did not scale as resources were underutilized, and it was expensive for organizations to maintain many physical servers.

Virtualized deployment era:

As a solution, virtualization was introduced. It allows you to run multiple Virtual Machines (VMs) on a single physical server's CPU. Virtualization allows applications to be isolated between VMs and provides a level of security as the information of one application cannot be freely accessed by another application.

Virtualization allows better utilization of resources in a physical server and allows better scalability because an application can be added or updated easily, reduces hardware costs, and much more. With virtualization you can present a set of physical resources as a cluster of disposable virtual machines.

Each VM is a full machine running all the components, including its own operating system, on top of the virtualized hardware.

Container deployment era:

Containers are similar to VMs, but they have relaxed isolation properties to share the Operating System (OS) among the applications. Therefore, containers are considered lightweight. Similar to a VM, a container has its own filesystem, share of CPU, memory, process space, and more. As they are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions.

Containers have become popular because they provide extra benefits, such as:

- Agile application creation and deployment: increased ease and efficiency of container image creation compared to VM image use.
- Continuous development, integration, and deployment: provides reliable and frequent container image build and deployment with quick and efficient rollbacks (due to image immutability).
- Dev and Ops separation of concerns: create application container images at build/release time rather than deployment time, thereby decoupling applications from infrastructure.
- Observability: not only surfaces OS-level information and metrics, but also application health and other signals.
- Environmental consistency across development, testing, and production: runs the same on a laptop as it does in the cloud.
- Cloud and OS distribution portability: runs on Ubuntu, RHEL, CoreOS, on-premises, on major public clouds, and anywhere else.
- Application-centric management: raises the level of abstraction from running an OS on virtual hardware to running an application on an OS using logical resources.
- Loosely coupled, distributed, elastic, liberated micro-services: applications are broken into smaller, independent pieces and can be deployed and managed dynamically – not a monolithic stack running on one big single-purpose machine.
- Resource isolation: predictable application performance.
- Resource utilization: high efficiency and density.

What's next

- Take a look at the Kubernetes Components
- Take a look at the The Kubernetes API
- Take a look at kubectl - the primary CLI for Kubernetes
- Take a look at the Cluster Architecture
- Ready to Get Started?

Feedback

Was this page helpful?

Yes No

Last modified March 01, 2026 at 2:32 PM PST: Add kubectl concept page to the overview section (2e10c1ef0e)

Cluster Architecture

Cluster Architecture

The architectural concepts behind Kubernetes.

A Kubernetes cluster consists of a control plane plus a set of worker machines, called nodes, that run containerized applications. Every cluster needs at least one worker node in order to run Pods.

The worker node(s) host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault-tolerance and high availability.

This document outlines the various components you need to have for a complete and working Kubernetes cluster.

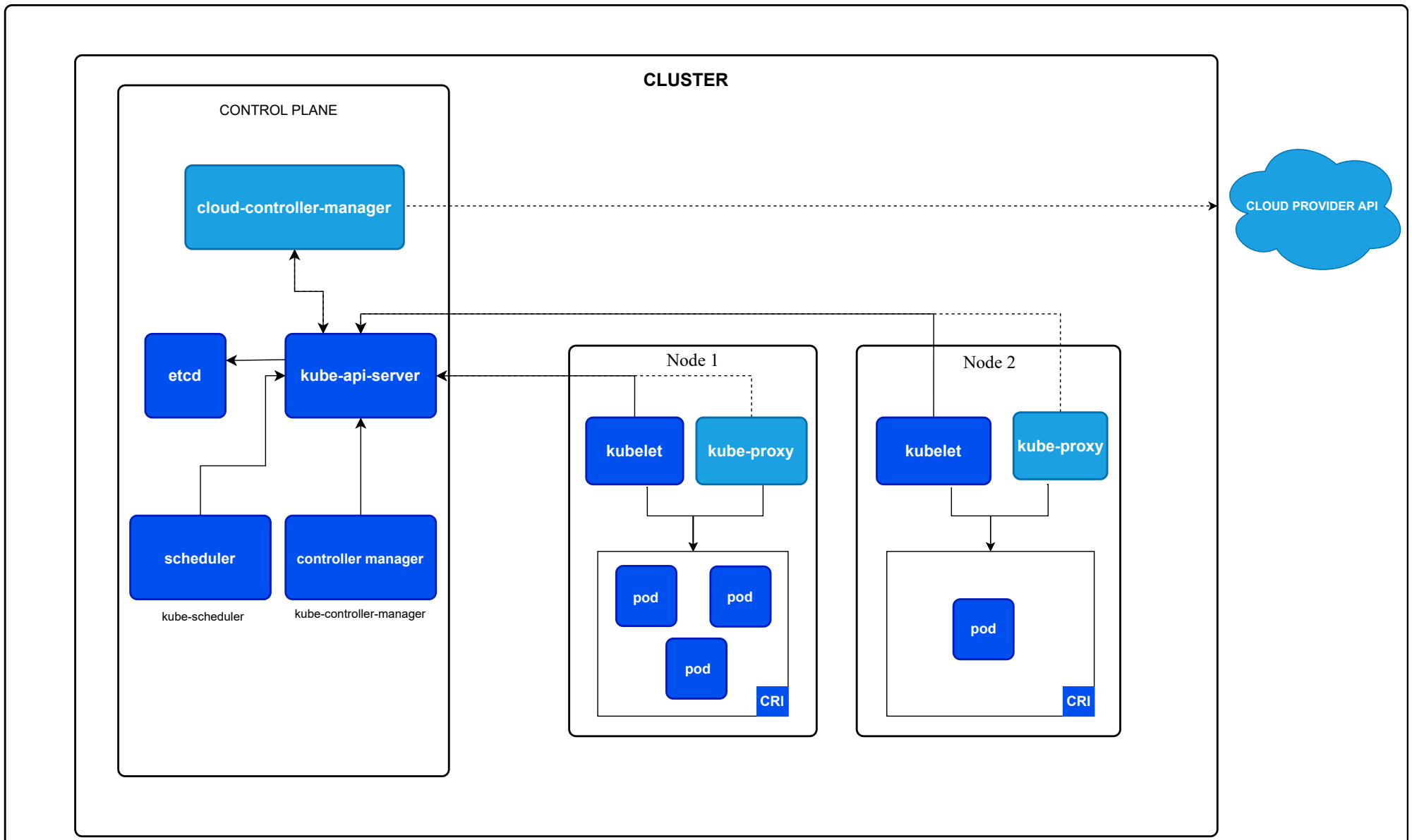


Figure 1. Kubernetes cluster components.

▶ About this architecture...

Control plane components

The control plane's components make global decisions about the cluster (for example, scheduling), as well as detecting and responding to cluster events (for example, starting up a new pod when a Deployment's `replicas` field is unsatisfied).

Control plane components can be run on any machine in the cluster. However, for simplicity, setup scripts typically start all control plane components on the same machine, and do not run user containers on this machine. See Creating Highly Available clusters with kubeadm for an example control plane setup that runs across multiple machines.

kube-apiserver

The API server is a component of the Kubernetes control plane that exposes the Kubernetes API. The API server is the front end for the Kubernetes control plane.

The main implementation of a Kubernetes API server is kube-apiserver. kube-apiserver is designed to scale horizontally—that is, it scales by deploying more instances. You can run several instances of kube-apiserver and balance traffic between those instances.

etcd

Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.

If your Kubernetes cluster uses etcd as its backing store, make sure you have a back up plan for the data.

You can find in-depth information about etcd in the official documentation.

kube-scheduler

Control plane component that watches for newly created Pods with no assigned node, and selects a node for them to run on.

Factors taken into account for scheduling decisions include: individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and deadlines.

kube-controller-manager

Control plane component that runs controller processes.

Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process.

There are many different types of controllers. Some examples of them are:

- Node controller: Responsible for noticing and responding when nodes go down.
- Job controller: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
- EndpointSlice controller: Populates EndpointSlice objects (to provide a link between Services and Pods).
- ServiceAccount controller: Create default ServiceAccounts for new namespaces.

The above is not an exhaustive list.

cloud-controller-manager

A Kubernetes control plane component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster.

The cloud-controller-manager only runs controllers that are specific to your cloud provider. If you are running Kubernetes on your own premises, or in a learning environment inside your own PC, the cluster does not have a cloud controller manager.

As with the kube-controller-manager, the cloud-controller-manager combines several logically independent control loops into a single binary that you run as a single process. You can scale horizontally (run more than one copy) to improve performance or to help tolerate failures.

The following controllers can have cloud provider dependencies:

- Node controller: For checking the cloud provider to determine if a node has been deleted in the cloud after it stops responding
- Route controller: For setting up routes in the underlying cloud infrastructure
- Service controller: For creating, updating and deleting cloud provider load balancers

---

Node components

Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment.

kubelet

An agent that runs on each node in the cluster. It makes sure that containers are running in a Pod.

The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. The kubelet doesn't manage containers which were not created by Kubernetes.

kube-proxy (optional)

kube-proxy is a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept.

kube-proxy maintains network rules on nodes. These network rules allow network communication to your Pods from network sessions inside or outside of your cluster.

kube-proxy uses the operating system packet filtering layer if there is one and it's available. Otherwise, kube-proxy forwards the traffic itself.

If you use a network plugin that implements packet forwarding for Services by itself, and providing equivalent behavior to kube-proxy, then you do not need to run kube-proxy on the nodes in your cluster.

Container runtime

A fundamental component that empowers Kubernetes to run containers effectively. It is responsible for managing the execution and lifecycle of containers within the Kubernetes environment.

Kubernetes supports container runtimes such as containerd, CRI-O, and any other implementation of the Kubernetes CRI (Container Runtime Interface).

Addons

Addons use Kubernetes resources (DaemonSet, Deployment, etc) to implement cluster features. Because these are providing cluster-level features, namespaced resources for addons belong within the `kube-system` namespace.

Selected addons are described below; for an extended list of available addons, please see Addons.

DNS

While the other addons are not strictly required, all Kubernetes clusters should have cluster DNS, as many examples rely on it.

Cluster DNS is a DNS server, in addition to the other DNS server(s) in your environment, which serves DNS records for Kubernetes services.

Containers started by Kubernetes automatically include this DNS server in their DNS searches.

Web UI (Dashboard)

Dashboard is a general purpose, web-based UI for Kubernetes clusters. It allows users to manage and troubleshoot applications running in the cluster, as well as the cluster itself.

Container resource monitoring

Container Resource Monitoring records generic time-series metrics about containers in a central database, and provides a UI for browsing that data.

Cluster-level Logging

A cluster-level logging mechanism is responsible for saving container logs to a central log store with a search/browsing interface.

Network plugins

Network plugins are software components that implement the container network interface (CNI) specification. They are responsible for allocating IP addresses to pods and enabling them to communicate with each other within the cluster.

Architecture variations

While the core components of Kubernetes remain consistent, the way they are deployed and managed can vary. Understanding these variations is crucial for designing and maintaining Kubernetes clusters that meet specific operational needs.

Control plane deployment options

The control plane components can be deployed in several ways:

Traditional deployment

Control plane components run directly on dedicated machines or VMs, often managed as systemd services.

Static Pods

Control plane components are deployed as static Pods, managed by the kubelet on specific nodes. This is a common approach used by tools like kubeadm.

Self-hosted

The control plane runs as Pods within the Kubernetes cluster itself, managed by Deployments and StatefulSets or other Kubernetes primitives.

Managed Kubernetes services

Cloud providers often abstract away the control plane, managing its components as part of their service offering.

Workload placement considerations

The placement of workloads, including the control plane components, can vary based on cluster size, performance requirements, and operational policies:

- In smaller or development clusters, control plane components and user workloads might run on the same nodes.
- Larger production clusters often dedicate specific nodes to control plane components, separating them from user workloads.
- Some organizations run critical add-ons or monitoring tools on control plane nodes.

Cluster management tools

Tools like kubeadm, kops, and Kubespray offer different approaches to deploying and managing clusters, each with its own method of component layout and management.

Customization and extensibility

Kubernetes architecture allows for significant customization:

- Custom schedulers can be deployed to work alongside the default Kubernetes scheduler or to replace it entirely.
- API servers can be extended with CustomResourceDefinitions and API Aggregation.
- Cloud providers can integrate deeply with Kubernetes using the cloud-controller-manager.

The flexibility of Kubernetes architecture allows organizations to tailor their clusters to specific needs, balancing factors such as operational complexity, performance, and management overhead.

What's next

Learn more about the following:

- Nodes and their communication with the control plane.
- Kubernetes controllers.
- Garbage collection of cluster objects.
- kube-scheduler which is the default scheduler for Kubernetes.
- Etcd's official documentation.
- Several container runtimes in Kubernetes.
- Integrating with cloud providers using cloud-controller-manager.

- kubectl commands.

Feedback

Was this page helpful?

Yes No

Last modified March 27, 2026 at 11:01 PM PST: fix architecture page orphan link (61fcd63f2a)

Pods

Pods

Pods are the smallest deployable units of computing that you can create and manage in Kubernetes.

A *Pod* (as in a pod of whales or pea pod) is a group of one or more containers, with shared storage and network resources, and a specification for how to run the containers. A Pod's contents are always co-located and co-scheduled, and run in a shared context. A Pod models an application-specific "logical host": it contains one or more application containers which are relatively tightly coupled. In non-cloud contexts, applications executed on the same physical or virtual machine are analogous to cloud applications executed on the same logical host.

As well as application containers, a Pod can contain init containers that run during Pod startup. You can also inject ephemeral containers for debugging a running Pod.

What is a Pod?

> **Note:**
> You need to install a container runtime into each node in the cluster so that Pods can run there.

The shared context of a Pod is a set of Linux namespaces, cgroups, and potentially other facets of isolation - the same things that isolate a container. Within a Pod's context, the individual applications may have further sub-isolations applied.

A Pod is similar to a set of containers with shared namespaces and shared filesystem volumes.

Pods in a Kubernetes cluster are used in two main ways:

- **Pods that run a single container**. The "one-container-per-Pod" model is the most common Kubernetes use case; in this case, you can think of a Pod as a wrapper around a single container; Kubernetes manages Pods rather than managing the containers directly.
- **Pods that run multiple containers that need to work together**. A Pod can encapsulate an application composed of multiple co-located containers that are tightly coupled and need to share resources. These co-located containers form a single cohesive unit.

  Grouping multiple co-located and co-managed containers in a single Pod is a relatively advanced use case. You should use this pattern only in specific instances in which your containers are tightly coupled.

  You don't need to run multiple containers to provide replication (for resilience or capacity); if you need multiple replicas, see Workload management.

Using Pods

The following is an example of a Pod which consists of a container running the image `nginx:1.14.2`.

pods/simple-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

To create the Pod shown above, run the following command:

```
kubectl apply -f https://k8s.io/examples/pods/simple-pod.yaml
```

Pods are generally not created directly and are created using workload resources. See Working with Pods for more information on how Pods are used with workload resources.

Workload resources for managing pods

Usually you don't need to create Pods directly, even singleton Pods. Instead, create them using workload resources such as Deployment or Job. If your Pods need to track state, consider the StatefulSet resource.

Each Pod is meant to run a single instance of a given application. If you want to scale your application horizontally (to provide more overall resources by running more instances), you should use multiple Pods, one for each instance. In Kubernetes, this is typically referred to as *replication*. Replicated Pods are usually created and managed as a group by a workload resource and its controller.

See Pods and controllers for more information on how Kubernetes uses workload resources, and their controllers, to implement application scaling and auto-healing.

Pods natively provide two kinds of shared resources for their constituent containers: networking and storage.

Working with Pods

You'll rarely create individual Pods directly in Kubernetes—even singleton Pods. This is because Pods are designed as relatively ephemeral, disposable entities. When a Pod gets created (directly by you, or indirectly by a controller), the new Pod is scheduled to run on a Node in your cluster. The Pod remains on that node until the Pod finishes execution, the Pod object is deleted, the Pod is *evicted* for lack of resources, or the node fails.

> **Note:**
> Restarting a container in a Pod should not be confused with restarting a Pod. A Pod is not a process, but an environment for running container(s). A Pod persists until it is deleted.

The name of a Pod must be a valid DNS subdomain value, but this can produce unexpected results for the Pod hostname. For best compatibility, the name should follow the more restrictive rules for a DNS label.

Pod OS

FEATURE STATE: Kubernetes v1.25 [stable]

You should set the `.spec.os.name` field to either `windows` or `linux` to indicate the OS on which you want the pod to run. These two are the only operating systems supported for now by Kubernetes. In the future, this list may be expanded.

In Kubernetes v1.36, the value of `.spec.os.name` does not affect how the kube-scheduler picks a node for the Pod to run on. In any cluster where there is more than one operating system for running nodes, you should set the kubernetes.io/os label correctly on each node, and define pods with a `nodeSelector` based on the operating system label. The kube-scheduler assigns your pod to a node based

on other criteria and may or may not succeed in picking a suitable node placement where the node OS is right for the containers in that Pod. The Pod security standards also use this field to avoid enforcing policies that aren't relevant to the operating system.

Pods and controllers

You can use workload resources to create and manage multiple Pods for you. A controller for the resource handles replication and rollout and automatic healing in case of Pod failure. For example, if a Node fails, a controller notices that Pods on that Node have stopped working and creates a replacement Pod. The scheduler places the replacement Pod onto a healthy Node.

Here are some examples of workload resources that manage one or more Pods:

- Deployment
- StatefulSet
- DaemonSet

Specifying a scheduling group

FEATURE STATE: Kubernetes v1.35 [alpha](disabled by default)

By default, Kubernetes schedules every Pod individually. However, some tightly-coupled applications need a group of Pods to be scheduled simultaneously to function correctly.

You can link a Pod to a PodGroup using the scheduling group field (`spec.schedulingGroup`). This tells the `kube-scheduler` that the `Pod` belongs to a specific group, enabling it to apply group-level coordinated placement decisions for the entire group at once.

Pod templates

Controllers for workload resources create Pods from a *pod template* and manage those Pods on your behalf.

PodTemplates are specifications for creating Pods, and are included in workload resources such as Deployments, Jobs, and DaemonSets.

Each controller for a workload resource uses the `PodTemplate` inside the workload object to make actual Pods. The `PodTemplate` is part of the desired state of whatever workload resource you used to run your app.

When you create a Pod, you can include environment variables in the Pod template for the containers that run in the Pod.

The sample below is a manifest for a simple Job with a `template` that starts one container. The container in that Pod prints a message then pauses.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello
spec:
  template:
    # This is the pod template
    spec:
      containers:
      - name: hello
        image: busybox:1.28
        command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 3600']
      restartPolicy: OnFailure
    # The pod template ends here
```

Modifying the pod template or switching to a new pod template has no direct effect on the Pods that already exist. If you change the pod template for a workload resource, that resource needs to create replacement Pods that use the updated template.

For example, the StatefulSet controller ensures that the running Pods match the current pod template for each StatefulSet object. If you edit the StatefulSet to change its pod template, the StatefulSet starts to create new Pods based on the updated template. Eventually, all of the old Pods are replaced with new Pods, and the update is complete.

Each workload resource implements its own rules for handling changes to the Pod template. If you want to read more about StatefulSet specifically, read Update strategy in the StatefulSet Basics tutorial.

On Nodes, the kubelet does not directly observe or manage any of the details around pod templates and updates; those details are abstracted away. That abstraction and separation of concerns simplifies system semantics, and makes it feasible to extend the cluster's behavior without changing existing code.

Pod update and replacement

As mentioned in the previous section, when the Pod template for a workload resource is changed, the controller creates new Pods based on the updated template instead of updating or patching the existing Pods.

Kubernetes doesn't prevent you from managing Pods directly. It is possible to update some fields of a running Pod, in place. However, Pod update operations like `patch`, and `replace` have some limitations:

- Most of the metadata about a Pod is immutable. For example, you cannot change the `namespace`, `name`, `uid`, or `creationTimestamp` fields.
- If the `metadata.deletionTimestamp` is set, no new entry can be added to the `metadata.finalizers` list.
- Pod updates may not change fields other than `spec.containers[*].image`, `spec.initContainers[*].image`, `spec.activeDeadlineSeconds`, `spec.terminationGracePeriodSeconds`, `spec.tolerations` or `spec.schedulingGates`. For `spec.tolerations`, you can only add new entries.
- When updating the `spec.activeDeadlineSeconds` field, two types of updates are allowed:
  1. setting the unassigned field to a positive number;
  2. updating the field from a positive number to a smaller, non-negative number.

Pod subresources

The above update rules apply to regular pod updates, but other pod fields can be updated through *subresources*.

- **Resize:** The `resize` subresource allows container resources (`spec.containers[*].resources`) to be updated. See Resize Container Resources for more details.
- **Ephemeral Containers:** The `ephemeralContainers` subresource allows ephemeral containers to be added to a Pod. See Ephemeral Containers for more details.
- **Status:** The `status` subresource allows the pod status to be updated. This is typically only used by the Kubelet and other system controllers.
- **Binding:** The `binding` subresource allows setting the pod's `spec.nodeName` via a `Binding` request. This is typically only used by the scheduler.

Pod generation

- The `metadata.generation` field is unique. It will be automatically set by the system such that new pods have a `metadata.generation` of 1, and every update to mutable fields in the pod's spec will increment the `metadata.generation` by 1.

ⓘ **FEATURE STATE:** Kubernetes v1.35 [stable](enabled by default)

- `observedGeneration` is a field that is captured in the `status` section of the Pod object. The Kubelet will set `status.observedGeneration` to track the pod state to the current pod status. The pod's `status.observedGeneration` will reflect the `metadata.generation` of the pod at the point that the pod status is being reported.

> **Note:**
> The `status.observedGeneration` field is managed by the kubelet and external controllers should **not** modify this field.

Different status fields may either be associated with the `metadata.generation` of the current sync loop, or with the `metadata.generation` of the previous sync loop. The key distinction is whether a change in the `spec` is reflected directly in the `status` or is an indirect result of a running process.

Direct Status Updates

For status fields where the allocated spec is directly reflected, the `observedGeneration` will be associated with the current `metadata.generation` (Generation N).

This behavior applies to:

- **Resize Status**: The status of a resource resize operation.
- **Allocated Resources**: The resources allocated to the Pod after a resize.
- **Ephemeral Containers**: When a new ephemeral container is added, and it is in `Waiting` state.

Indirect Status Updates

For status fields that are an indirect result of running the spec, the `observedGeneration` will be associated with the `metadata.generation` of the previous sync loop (Generation N-1).

This behavior applies to:

- **Container Image**: The `ContainerStatus.ImageID` reflects the image from the previous generation until the new image is pulled and the container is updated.
- **Actual Resources**: During an in-progress resize, the actual resources in use still belong to the previous generation's request.
- **Container state**: During an in-progress resize, with require restart policy reflects the previous generation's request.
- **activeDeadlineSeconds** & **terminationGracePeriodSeconds** & **deletionTimestamp**: The effects of these fields on the Pod's status are a result of the previously observed specification.

Resource sharing and communication

Pods enable data sharing and communication among their constituent containers.

Storage in Pods

A Pod can specify a set of shared storage volumes. All containers in the Pod can access the shared volumes, allowing those containers to share data. Volumes also allow persistent data in a Pod to survive in case one of the containers within needs to be restarted. See Storage for more information on how Kubernetes implements shared storage and makes it available to Pods.

Pod networking

Each Pod is assigned a unique IP address for each address family. Every container in a Pod shares the network namespace, including the IP address and network ports. Inside a Pod (and **only** then), the containers that belong to the Pod can communicate with one another using `localhost`. When containers in a Pod communicate with entities *outside the Pod*, they must coordinate how they use the shared network resources (such as ports). Within a Pod, containers share an IP address and port space, and can find each other via `localhost`. The containers in a Pod can also communicate with each other using standard inter-process communications like SystemV semaphores or POSIX shared memory. Containers in different Pods have distinct IP addresses and can not communicate by OS-level IPC without special configuration. Containers that want to interact with a container running in a different Pod can use IP networking to communicate.

Containers within the Pod see the system hostname as being the same as the configured `name` for the Pod. There's more about this in the networking section.

Pod security settings

To set security constraints on Pods and containers, you use the `securityContext` field in the Pod specification. This field gives you granular control over what a Pod or individual containers can do. See Advanced Pod Configuration for more details.

For basic security configuration, you should meet the Baseline Pod security standard and run containers as non-root. You can set simple security contexts:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  containers:
  - name: sec-ctx-demo
    image: busybox
    command: ["sh", "-c", "sleep 1h"]
```

For advanced security context configuration including capabilities, seccomp profiles, and detailed security options, see the security concepts section.

- To learn about kernel-level security constraints that you can use, see Linux kernel security constraints for Pods and containers.
- To learn more about the Pod security context, see Configure a Security Context for a Pod or Container.

Resource requests and limits

When you specify a Pod, you can optionally specify how much of each resource a container needs. The most common resources to specify are CPU and memory (RAM).

When you specify the resource *request* for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on. When you specify a resource *limit* for a container, the kubelet enforces those limits so that the running container is not allowed to use more of that resource than the limit you set.

CPU limits are enforced by CPU throttling. When a container approaches its CPU limit, the kernel restricts its access to CPU. Memory limits are enforced by the kernel with out-of-memory (OOM) kills when a container exceeds its limit.

> **Note:**
> Setting CPU limits involves a trade-off. CPU limits help prevent noisy neighbor problems where a single workload starves others on the same node. This is especially important in multi-tenant environments. However, CPU limits can cause throttling even when the node has spare CPU capacity, potentially degrading latency-sensitive workload performance. Whether to set CPU limits depends on your environment, workload characteristics, and isolation requirements.

For details on resource units, enforcement behavior, and configuration examples, see Resource Management for Pods and Containers.

Static Pods

Static Pods are managed directly by the kubelet daemon on a specific node, without the API server observing them. Whereas most Pods are managed by the control plane (for example, a Deployment), for static Pods, the kubelet directly supervises each static Pod (and restarts it if it fails).

Static Pods are always bound to one Kubelet on a specific node. The main use for static Pods is to run a self-hosted control plane: in other words, using the kubelet to supervise the individual control plane components.

The kubelet automatically tries to create a mirror Pod on the Kubernetes API server for each static Pod. This means that the Pods running on a node are visible on the API server, but cannot be controlled from there. See the guide Create static Pods for more information.

> **Note:**
> The `spec` of a static Pod cannot refer to other API objects (e.g., ServiceAccount, ConfigMap, Secret, etc).

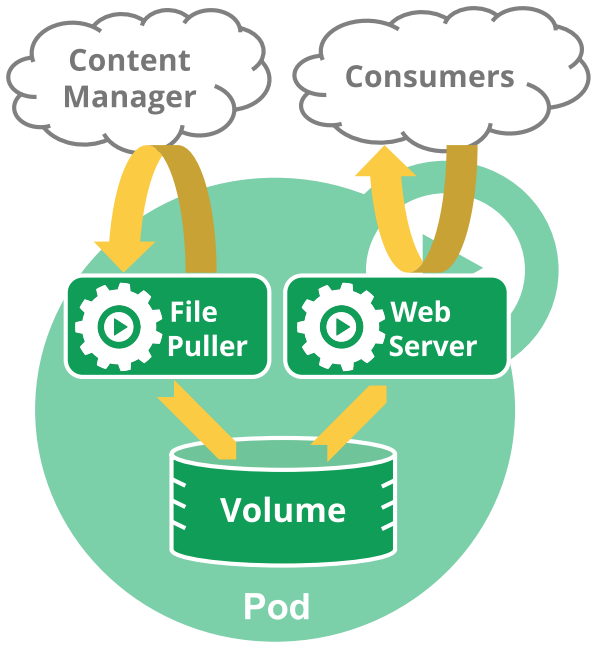
Pods with multiple containers

Pods are designed to support multiple cooperating processes (as containers) that form a cohesive unit of service. The containers in a Pod are automatically co-located and co-scheduled on the same physical or virtual machine in the cluster. The containers can share resources and dependencies, communicate with one another, and coordinate when and how they are terminated.

Pods in a Kubernetes cluster are used in two main ways:

- **Pods that run a single container**. The "one-container-per-Pod" model is the most common Kubernetes use case; in this case, you can think of a Pod as a wrapper around a single container; Kubernetes manages Pods rather than managing the containers directly.
- **Pods that run multiple containers that need to work together**. A Pod can encapsulate an application composed of multiple co-located containers that are tightly coupled and need to share resources. These co-located containers form a single cohesive unit of service—for example, one container serving data stored in a shared volume to the public, while a separate sidecar container refreshes or updates those files. The Pod wraps these containers, storage resources, and an ephemeral network identity together as a single unit.

For example, you might have a container that acts as a web server for files in a shared volume, and a separate sidecar container that updates those files from a remote source, as in the following diagram:



Some Pods have init containers as well as app containers. By default, init containers run and complete before the app containers are started.

You can also have sidecar containers that provide auxiliary services to the main application Pod (for example: a service mesh).

ⓘ **FEATURE STATE:** Kubernetes v1.33 [stable](enabled by default)

Enabled by default, the `SidecarContainers` feature gate allows you to specify `restartPolicy: Always` for init containers. Setting the `Always` restart policy ensures that the containers where you set it are treated as *sidecars* that are kept running during the entire lifetime of the Pod. Containers that you explicitly define as sidecar containers start up before the main application Pod and remain running until the Pod is shut down.

Container probes

A *probe* is a diagnostic performed periodically by the kubelet on a container. To perform a diagnostic, the kubelet can invoke different actions:

- `ExecAction` (performed with the help of the container runtime)
- `TCPSocketAction` (checked directly by the kubelet)
- `HTTPGetAction` (checked directly by the kubelet)

You can read more about probes in the Pod Lifecycle documentation.

What's next

- Learn about the lifecycle of a Pod.
- Read about PodDisruptionBudget and how you can use it to manage application availability during disruptions.

- Pod is a top-level resource in the Kubernetes REST API. The Pod object definition describes the object in detail.
- The Distributed System Toolkit: Patterns for Composite Containers explains common layouts for Pods with more than one container.
- Read about Pod topology spread constraints
- Read Advanced Pod Configuration to learn the topic in detail. That page covers aspects of Pod configuration beyond the essentials, including:
  - PriorityClasses
  - RuntimeClasses
  - advanced ways to configure *scheduling*: the way that Kubernetes decides which node a Pod should run on.

To understand the context for why Kubernetes wraps a common Pod API in other resources (such as StatefulSets or Deployments), you can read about the prior art, including:

- Aurora
- Borg
- Marathon
- Omega
- Tupperware.

Feedback

Was this page helpful?

Yes No

Last modified March 26, 2026 at 7:39 PM PST: Update Pod reference for PodGroup (e77da46210)

Deployments

Deployments

A Deployment manages a set of Pods to run an application workload, usually one that doesn't maintain state.

A *Deployment* provides declarative updates for Pods and ReplicaSets.

You describe a *desired state* in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate. You can define Deployments to create new ReplicaSets, or to remove existing Deployments and adopt all their resources with new Deployments.

> **Note:**
> Do not manage ReplicaSets owned by a Deployment. Consider opening an issue in the main Kubernetes repository if your use case is not covered below.

Use Case

The following are typical use cases for Deployments:

- Create a Deployment to rollout a ReplicaSet. The ReplicaSet creates Pods in the background. Check the status of the rollout to see if it succeeds or not.
- Declare the new state of the Pods by updating the PodTemplateSpec of the Deployment. A new ReplicaSet is created, and the Deployment gradually scales it up while scaling down the old ReplicaSet, ensuring Pods are replaced at a controlled rate. Each new ReplicaSet updates the revision of the Deployment.
- Rollback to an earlier Deployment revision if the current state of the Deployment is not stable. Each rollback updates the revision of the Deployment.
- Scale up the Deployment to facilitate more load.
- Pause the rollout of a Deployment to apply multiple fixes to its PodTemplateSpec and then resume it to start a new rollout.
- Use the status of the Deployment as an indicator that a rollout has stuck.
- Clean up older ReplicaSets that you don't need anymore.

Creating a Deployment

The following is an example of a Deployment. It creates a ReplicaSet to bring up three `nginx` Pods:

controllers/nginx-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

In this example:

- A Deployment named `nginx-deployment` is created, indicated by the `.metadata.name` field. This name will become the basis for the ReplicaSets and Pods which are created later. See Writing a Deployment Spec for more details.
- The Deployment creates a ReplicaSet that creates three replicated Pods, indicated by the `.spec.replicas` field.
- The `.spec.selector` field defines how the created ReplicaSet finds which Pods to manage. In this case, you select a label that is defined in the Pod template (`app: nginx`). However, more sophisticated selection rules are possible, as long as the Pod template itself satisfies the rule.

> **Note:**
> The `.spec.selector.matchLabels` field is a map of {key,value} pairs. A single {key,value} in the `matchLabels` map is equivalent to an element of `matchExpressions`, whose `key` field is "key", the `operator` is "In", and the `values` array contains only "value". All of the requirements, from both `matchLabels` and `matchExpressions`, must be satisfied in order to match.

- The `.spec.template` field contains the following sub-fields:
  - The Pods are labeled `app: nginx` using the `.metadata.labels` field.
  - The Pod template's specification, or `.spec` field, indicates that the Pods run one container, `nginx` , which runs the `nginx` Docker Hub image at version 1.14.2.
  - Create one container and name it `nginx` using the `.spec.containers[0].name` field.

Before you begin, make sure your Kubernetes cluster is up and running. Follow the steps given below to create the above Deployment:

1. Create the Deployment by running the following command:

```
kubectl apply -f https://k8s.io/examples/controllers/nginx-deployment.yaml
```

2. Run `kubectl get deployments` to check if the Deployment was created.

   If the Deployment is still being created, the output is similar to the following:

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   0/3     0            0           1s
```

   When you inspect the Deployments in your cluster, the following fields are displayed:

- `NAME` lists the names of the Deployments in the namespace.
- `READY` displays how many replicas of the application are available to your users. It follows the pattern ready/desired.
- `UP-TO-DATE` displays the number of replicas that have been updated to achieve the desired state.
- `AVAILABLE` displays how many replicas of the application are available to your users.
- `AGE` displays the amount of time that the application has been running.

Notice how the number of desired replicas is 3 according to `.spec.replicas` field.

3. To see the Deployment rollout status, run `kubectl rollout status deployment/nginx-deployment .`

   The output is similar to:

```
Waiting for rollout to finish: 2 out of 3 new replicas have been updated...
deployment "nginx-deployment" successfully rolled out
```

4. Run the `kubectl get deployments` again a few seconds later. The output is similar to this:

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           18s
```

   Notice that the Deployment has created all three replicas, and all replicas are up-to-date (they contain the latest Pod template) and available.

5. To see the ReplicaSet (`rs`) created by the Deployment, run `kubectl get rs` . The output is similar to this:

```
NAME                          DESIRED   CURRENT   READY   AGE
nginx-deployment-75675f5897   3         3         3       18s
```

   ReplicaSet output shows the following fields:

   - `NAME` lists the names of the ReplicaSets in the namespace.
   - `DESIRED` displays the desired number of *replicas* of the application, which you define when you create the Deployment. This is the *desired state*.
   - `CURRENT` displays how many replicas are currently running.
   - `READY` displays how many replicas of the application are available to your users.
   - `AGE` displays the amount of time that the application has been running.

   Notice that the name of the ReplicaSet is always formatted as `[DEPLOYMENT-NAME]-[HASH]` . This name will become the basis for the Pods which are created.

   The `HASH` string is the same as the `pod-template-hash` label on the ReplicaSet.

6. To see the labels automatically generated for each Pod, run `kubectl get pods --show-labels` . The output is similar to:

```
NAME                                READY     STATUS    RESTARTS   AGE       LABELS
nginx-deployment-75675f5897-7ci7o   1/1       Running   0          18s       app=nginx,pod-template-hash=75675f5897
nginx-deployment-75675f5897-kzszj   1/1       Running   0          18s       app=nginx,pod-template-hash=75675f5897
nginx-deployment-75675f5897-qqcnn   1/1       Running   0          18s       app=nginx,pod-template-hash=75675f5897
```

   The created ReplicaSet ensures that there are three `nginx` Pods.

> **Note:**
> You must specify an appropriate selector and Pod template labels in a Deployment (in this case, `app: nginx`).
>
> Do not overlap labels or selectors with other controllers (including other Deployments and StatefulSets). Kubernetes doesn't stop you from overlapping, and if multiple controllers have overlapping selectors those controllers might conflict and behave unexpectedly.

Pod-template-hash label

> **Caution:**
> Do not change this label.

The `pod-template-hash` label is added by the Deployment controller to every ReplicaSet that a Deployment creates or adopts.

This label ensures that child ReplicaSets of a Deployment do not overlap. It is generated by hashing the `PodTemplate` of the ReplicaSet and using the resulting hash as the label value that is added to the ReplicaSet selector, Pod template labels, and in any existing Pods that the ReplicaSet might have.

Updating a Deployment

> **Note:**
> A Deployment's rollout is triggered if and only if the Deployment's Pod template (that is, `.spec.template`) is changed, for example if the labels or container images of the template are updated. Other updates, such as scaling the Deployment, do not trigger a rollout.

Follow the steps given below to update your Deployment:

1. Let's update the nginx Pods to use the `nginx:1.16.1` image instead of the `nginx:1.14.2` image.

   ```
   kubectl set image deployment.v1.apps/nginx-deployment nginx=nginx:1.16.1
   ```

   or use the following command:

   ```
   kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1
   ```

   where `deployment/nginx-deployment` indicates the Deployment, `nginx` indicates the Container the update will take place and `nginx:1.16.1` indicates the new image and its tag.

   The output is similar to:

   ```
   deployment.apps/nginx-deployment image updated
   ```

   Alternatively, you can `edit` the Deployment and change `.spec.template.spec.containers[0].image` from `nginx:1.14.2` to `nginx:1.16.1`:

   ```
   kubectl edit deployment/nginx-deployment
   ```

   The output is similar to:

   ```
   deployment.apps/nginx-deployment edited
   ```

2. To see the rollout status, run:

   ```
   kubectl rollout status deployment/nginx-deployment
   ```

   The output is similar to this:

```
Waiting for rollout to finish: 2 out of 3 new replicas have been updated...
```

or

```
deployment "nginx-deployment" successfully rolled out
```

Get more details on your updated Deployment:

- After the rollout succeeds, you can view the Deployment by running `kubectl get deployments`. The output is similar to this:

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           36s
```

- Run `kubectl get rs` to see that the Deployment updated the Pods by creating a new ReplicaSet and scaling it up to 3 replicas, as well as scaling down the old ReplicaSet to 0 replicas.

```
kubectl get rs
```

The output is similar to this:

```
NAME                          DESIRED   CURRENT   READY   AGE
nginx-deployment-1564180365   3         3         3       6s
nginx-deployment-2035384211   0         0         0       36s
```

- Running `get pods` should now show only the new Pods:

```
kubectl get pods
```

The output is similar to this:

```
NAME                               READY     STATUS    RESTARTS   AGE
nginx-deployment-1564180365-khku8   1/1       Running   0          14s
nginx-deployment-1564180365-nacti   1/1       Running   0          14s
nginx-deployment-1564180365-z9gth   1/1       Running   0          14s
```

Next time you want to update these Pods, you only need to update the Deployment's Pod template again.

Deployment ensures that only a certain number of Pods are down while they are being updated. By default, it ensures that at least 75% of the desired number of Pods are up (25% max unavailable).

Deployment also ensures that only a certain number of Pods are created above the desired number of Pods. By default, it ensures that at most 125% of the desired number of Pods are up (25% max surge).

For example, if you look at the above Deployment closely, you will see that it first creates a new Pod, then deletes an old Pod, and creates another new one. It does not kill old Pods until a sufficient number of new Pods have come up, and does not create new Pods until a sufficient number of old Pods have been killed. It makes sure that at least 3 Pods are available and that at max 4 Pods in total are available. In case of a Deployment with 4 replicas, the number of Pods would be between 3 and 5.

- Get details of your Deployment:

```
kubectl describe deployments
```

The output is similar to this:

```
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Thu, 30 Nov 2017 10:56:25 +0000
Labels:                 app=nginx
Annotations:            deployment.kubernetes.io/revision=2
Selector:               app=nginx
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
   Containers:
    nginx:
      Image:        nginx:1.16.1
      Port:         80/TCP
      Environment:  <none>
      Mounts:       <none>
    Volumes:        <none>
  Conditions:
    Type           Status  Reason
    ----           ------  ------
    Available      True    MinimumReplicasAvailable
    Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-1564180365 (3/3 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  2m    deployment-controller  Scaled up replica set nginx-deployment-2035384211 to 3
  Normal  ScalingReplicaSet  24s   deployment-controller  Scaled up replica set nginx-deployment-1564180365 to 1
  Normal  ScalingReplicaSet  22s   deployment-controller  Scaled down replica set nginx-deployment-2035384211 to 2
  Normal  ScalingReplicaSet  22s   deployment-controller  Scaled up replica set nginx-deployment-1564180365 to 2
  Normal  ScalingReplicaSet  19s   deployment-controller  Scaled down replica set nginx-deployment-2035384211 to 1
  Normal  ScalingReplicaSet  19s   deployment-controller  Scaled up replica set nginx-deployment-1564180365 to 3
  Normal  ScalingReplicaSet  14s   deployment-controller  Scaled down replica set nginx-deployment-2035384211 to 0
```

Here you see that when you first created the Deployment, it created a ReplicaSet (nginx-deployment-2035384211) and scaled it up to 3 replicas directly. When you updated the Deployment, it created a new ReplicaSet (nginx-deployment-1564180365) and scaled it up to 1 and waited for it to come up. Then it scaled down the old ReplicaSet to 2 and scaled up the new ReplicaSet to 2 so that at least 3 Pods were available and at most 4 Pods were created at all times. It then continued scaling up and down the new and the old ReplicaSet, with the same rolling update strategy. Finally, you'll have 3 available replicas in the new ReplicaSet, and the old ReplicaSet is scaled down to 0.

> **Note:**
> Kubernetes doesn't count terminating Pods when calculating the number of `availableReplicas`, which must be between `replicas - maxUnavailable` and `replicas + maxSurge`. As a result, you might notice that there are more Pods than expected during a rollout, and that the total resources consumed by the Deployment is more than `replicas + maxSurge` until the `terminationGracePeriodSeconds` of the terminating Pods expires.

Rollover (aka multiple updates in-flight)

Each time a new Deployment is observed by the Deployment controller, a ReplicaSet is created to bring up the desired Pods. If the Deployment is updated, the existing ReplicaSet that controls Pods whose labels match `.spec.selector` but whose template does not match `.spec.template` is scaled down. Eventually, the new ReplicaSet is scaled to `.spec.replicas` and all old ReplicaSets is scaled to 0.

If you update a Deployment while an existing rollout is in progress, the Deployment creates a new ReplicaSet as per the update and start scaling that up, and rolls over the ReplicaSet that it was scaling up previously -- it will add it to its list of old ReplicaSets and start scaling it down.

For example, suppose you create a Deployment to create 5 replicas of `nginx:1.14.2`, but then update the Deployment to create 5 replicas of `nginx:1.16.1`, when only 3 replicas of `nginx:1.14.2` had been created. In that case, the Deployment immediately starts killing the 3 `nginx:1.14.2` Pods that it had created, and starts creating `nginx:1.16.1` Pods. It does not wait for the 5 replicas of `nginx:1.14.2` to be created before changing course.

Label selector updates

It is generally discouraged to make label selector updates and it is suggested to plan your selectors up front. A Deployment's label selector is **immutable** after creation; it cannot be updated via `kubectl patch`, `kubectl edit`, `kubectl apply`, or tools like `helm upgrade`.

If you must change the selector, you have to delete the Deployment and recreate it. Exercise great caution and ensure you grasp the following implications:

- **Additions:** When you create a new Deployment with a narrower selector, the new Deployment **must** also have a suitable Pod template. If you have an existing manifest and you edit the manifest to narrow the selector, you need to edit the metadata of the Pod template inside that Deployment, adding the new labels to match, as otherwise the API server returns a validation error. This is a *non-overlapping* change: the new Deployment will not "see" the old Pods (which lack the new label), causing the old ReplicaSet to be **orphaned** and a brand-new ReplicaSet to be created.
- **Value Updates:** Changing the existing value in a selector key (e.g., from `v1` to `v2`) results in the same behavior as additions (orphaning and recreation).
- **Removals:** Removing an existing key from the Deployment selector does not require any changes in the Pod template labels. This is an *overlapping* change: the new, broader selector would match the old Pods. Existing ReplicaSets are not orphaned, and a new ReplicaSet is not created, but note that the removed label still exists in any existing Pods and ReplicaSets. You can clean that up by triggering a rollout for the Deployment.

Rolling Back a Deployment

Sometimes, you may want to rollback a Deployment; for example, when the Deployment is not stable, such as crash looping. By default, all of the Deployment's rollout history is kept in the system so that you can rollback anytime you want (you can change that by modifying revision history limit).

> **Note:**
> A Deployment's revision is created when a Deployment's rollout is triggered. This means that the new revision is created if and only if the Deployment's Pod template (`.spec.template`) is changed, for example if you update the labels or container images of the template. Other updates, such as scaling the Deployment, do not create a Deployment revision, so that you can facilitate simultaneous manual- or auto-scaling. This means that when you roll back to an earlier revision, only the Deployment's Pod template part is rolled back.

- Suppose that you made a typo while updating the Deployment, by putting the image name as `nginx:1.161` instead of `nginx:1.16.1`:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.161
```

  The output is similar to this:

```
deployment.apps/nginx-deployment image updated
```

- The rollout gets stuck. You can verify it by checking the rollout status:

```
kubectl rollout status deployment/nginx-deployment
```

  The output is similar to this:

```
Waiting for rollout to finish: 1 out of 3 new replicas have been updated...
```

- Press Ctrl-C to stop the above rollout status watch. For more information on stuck rollouts, read more here.
- You see that the number of old replicas (adding the replica count from `nginx-deployment-1564180365` and `nginx-deployment-2035384211`) is 3, and the number of new replicas (from `nginx-deployment-3066724191`) is 1.

```
kubectl get rs
```

The output is similar to this:

```
NAME                          DESIRED   CURRENT   READY   AGE
nginx-deployment-1564180365   3         3         3       25s
nginx-deployment-2035384211   0         0         0       36s
nginx-deployment-3066724191   1         1         0       6s
```

- Looking at the Pods created, you see that 1 Pod created by new ReplicaSet is stuck in an image pull loop.

```
kubectl get pods
```

The output is similar to this:

```
NAME                                READY     STATUS             RESTARTS   AGE
nginx-deployment-1564180365-70iae   1/1       Running            0          25s
nginx-deployment-1564180365-jbqqo   1/1       Running            0          25s
nginx-deployment-1564180365-hysrc   1/1       Running            0          25s
nginx-deployment-3066724191-08mng   0/1       ImagePullBackOff   0          6s
```

> **Note:**
> The Deployment controller stops the bad rollout automatically, and stops scaling up the new ReplicaSet. This depends on the rollingUpdate parameters (`maxUnavailable` specifically) that you have specified. Kubernetes by default sets the value to 25%.

- Get the description of the Deployment:

```
kubectl describe deployment
```

The output is similar to this:

```
Name:           nginx-deployment
Namespace:      default
CreationTimestamp:  Tue, 15 Mar 2016 14:48:04 -0700
Labels:         app=nginx
Selector:       app=nginx
Replicas:       3 desired | 1 updated | 4 total | 3 available | 1 unavailable
StrategyType:       RollingUpdate
MinReadySeconds:    0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
   nginx:
    Image:        nginx:1.161
    Port:         80/TCP
    Host Port:    0/TCP
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    ReplicaSetUpdated
OldReplicaSets:     nginx-deployment-1564180365 (3/3 replicas created)
NewReplicaSet:      nginx-deployment-3066724191 (1/1 replicas created)
Events:
  FirstSeen LastSeen    Count   From                    SubObjectPath   Type        Reason              Message
  --------- --------    -----   ----                    -------------   --------    ------              -------
  1m        1m          1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled up replica set
  22s       22s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled up replica set
  22s       22s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled down replica s
  22s       22s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled up replica set
  21s       21s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled down replica s
  21s       21s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled up replica set
  13s       13s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled down replica s
  13s       13s         1       {deployment-controller }                Normal      ScalingReplicaSet   Scaled up replica set
```

To fix this, you need to rollback to a previous revision of Deployment that is stable.

Checking Rollout History of a Deployment

Follow the steps given below to check the rollout history:

1. First, check the revisions of this Deployment:

```
kubectl rollout history deployment/nginx-deployment
```

The output is similar to this:

```
deployments "nginx-deployment"
REVISION    CHANGE-CAUSE
1           <none>
2           <none>
3           <none>
```

`CHANGE-CAUSE` is copied from the Deployment annotation `kubernetes.io/change-cause` to its revisions upon creation. You can specify the `CHANGE-CAUSE` message by:

- Annotating the Deployment with `kubectl annotate deployment/nginx-deployment kubernetes.io/change-cause="image updated to 1.16.1"`
- Manually editing the manifest of the resource.
- Using tooling that sets the annotation automatically.

> **Note:**
> In older versions of Kubernetes, you could use the `--record` flag with kubectl commands to automatically populate the `CHANGE-CAUSE` field. This flag is deprecated and will be removed in a future release.

2. To see the details of each revision, run:

```
kubectl rollout history deployment/nginx-deployment --revision=2
```

The output is similar to this:

```
deployments "nginx-deployment" revision 2
  Labels:       app=nginx
          pod-template-hash=1159050644
  Containers:
   nginx:
    Image:      nginx:1.16.1
    Port:       80/TCP
     QoS Tier:
        cpu:      BestEffort
        memory:   BestEffort
    Environment Variables:      <none>
  No volumes.
```

Rolling Back to a Previous Revision

Follow the steps given below to rollback the Deployment from the current version to the previous version, which is version 2.

1. Now you've decided to undo the current rollout and rollback to the previous revision:

```
kubectl rollout undo deployment/nginx-deployment
```

The output is similar to this:

```
deployment.apps/nginx-deployment rolled back
```

Alternatively, you can rollback to a specific revision by specifying it with `--to-revision`:

```
kubectl rollout undo deployment/nginx-deployment --to-revision=2
```

The output is similar to this:

```
deployment.apps/nginx-deployment rolled back
```

For more details about rollout related commands, read `kubectl rollout`.

The Deployment is now rolled back to a previous stable revision. As you can see, a `DeploymentRollback` event for rolling back to revision 2 is generated from Deployment controller.

2. Check if the rollback was successful and the Deployment is running as expected, run:

```
kubectl get deployment nginx-deployment
```

The output is similar to this:

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           30m
```

3. Get the description of the Deployment:

```
kubectl describe deployment nginx-deployment
```

The output is similar to this:

```
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Sun, 02 Sep 2018 18:17:55 -0500
Labels:                 app=nginx
Annotations:            deployment.kubernetes.io/revision=4
Selector:               app=nginx
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
   nginx:
    Image:        nginx:1.16.1
    Port:         80/TCP
    Host Port:    0/TCP
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-c4747d96c (3/3 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  12m   deployment-controller  Scaled up replica set nginx-deployment-75675f5897 to 3
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set nginx-deployment-c4747d96c to 1
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled down replica set nginx-deployment-75675f5897 to 2
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set nginx-deployment-c4747d96c to 2
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled down replica set nginx-deployment-75675f5897 to 1
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set nginx-deployment-c4747d96c to 3
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled down replica set nginx-deployment-75675f5897 to 0
  Normal  ScalingReplicaSet  11m   deployment-controller  Scaled up replica set nginx-deployment-595696685f to 1
  Normal  DeploymentRollback 15s   deployment-controller  Rolled back deployment "nginx-deployment" to revision 2
  Normal  ScalingReplicaSet  15s   deployment-controller  Scaled down replica set nginx-deployment-595696685f to 0
```

Scaling a Deployment

You can scale a Deployment by using the following command:

```
kubectl scale deployment/nginx-deployment --replicas=10
```

The output is similar to this:

```
deployment.apps/nginx-deployment scaled
```

Assuming horizontal Pod autoscaling is enabled in your cluster, you can set up an autoscaler for your Deployment and choose the minimum and maximum number of Pods you want to run based on the CPU utilization of your existing Pods.

```
kubectl autoscale deployment/nginx-deployment --min=10 --max=15 --cpu-percent=80%
```

The output is similar to this:

```
deployment.apps/nginx-deployment scaled
```

Proportional scaling

RollingUpdate Deployments support running multiple versions of an application at the same time. When you or an autoscaler scales a RollingUpdate Deployment that is in the middle of a rollout (either in progress or paused), the Deployment controller balances the additional replicas in the existing active ReplicaSets (ReplicaSets with Pods) in order to mitigate risk. This is called *proportional scaling*.

For example, you are running a Deployment with 10 replicas, maxSurge=3, and maxUnavailable=2.

- Ensure that the 10 replicas in your Deployment are running.

  ```
  kubectl get deploy
  ```

  The output is similar to this:

  ```
  NAME                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
  nginx-deployment    10        10        10           10          50s
  ```

- You update to a new image which happens to be unresolvable from inside the cluster.

  ```
  kubectl set image deployment/nginx-deployment nginx=nginx:sometag
  ```

  The output is similar to this:

  ```
  deployment.apps/nginx-deployment image updated
  ```

- The image update starts a new rollout with ReplicaSet nginx-deployment-1989198191, but it's blocked due to the `maxUnavailable` requirement that you mentioned above. Check out the rollout status:

  ```
  kubectl get rs
  ```

  The output is similar to this:

  ```
  NAME                          DESIRED   CURRENT   READY     AGE
  nginx-deployment-1989198191   5         5         0         9s
  nginx-deployment-618515232    8         8         8         1m
  ```

- Then a new scaling request for the Deployment comes along. The autoscaler increments the Deployment replicas to 15. The Deployment controller needs to decide where to add these new 5 replicas. If you weren't using proportional scaling, all 5 of them would be added in the new ReplicaSet. With proportional scaling, you spread the additional replicas across all ReplicaSets. Bigger proportions go to the ReplicaSets with the most replicas and lower proportions go to ReplicaSets with less replicas. Any leftovers are added to the ReplicaSet with the most replicas. ReplicaSets with zero replicas are not scaled up.

In our example above, 3 replicas are added to the old ReplicaSet and 2 replicas are added to the new ReplicaSet. The rollout process should eventually move all replicas to the new ReplicaSet, assuming the new replicas become healthy. To confirm this, run:

```
kubectl get deploy
```

The output is similar to this:

```
NAME                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    15        18        7            8           7m
```

The rollout status confirms how the replicas were added to each ReplicaSet.

```
kubectl get rs
```

The output is similar to this:

```
NAME                          DESIRED   CURRENT   READY     AGE
nginx-deployment-1989198191   7         7         0         7m
nginx-deployment-618515232    11        11        11        7m
```

Pausing and Resuming a rollout of a Deployment

When you update a Deployment, or plan to, you can pause rollouts for that Deployment before you trigger one or more updates. When you're ready to apply those changes, you resume rollouts for the Deployment. This approach allows you to apply multiple fixes in between pausing and resuming without triggering unnecessary rollouts.

- For example, with a Deployment that was created:

  Get the Deployment details:

  ```
  kubectl get deploy
  ```

  The output is similar to this:

  ```
  NAME      DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
  nginx     3         3         3            3           1m
  ```

  Get the rollout status:

  ```
  kubectl get rs
  ```

  The output is similar to this:

  ```
  NAME               DESIRED   CURRENT   READY     AGE
  nginx-2142116321   3         3         3         1m
  ```

- Pause by running the following command:

  ```
  kubectl rollout pause deployment/nginx-deployment
  ```

  The output is similar to this:

```
deployment.apps/nginx-deployment paused
```

- Then update the image of the Deployment:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1
```

The output is similar to this:

```
deployment.apps/nginx-deployment image updated
```

- Notice that no new rollout started:

```
kubectl rollout history deployment/nginx-deployment
```

The output is similar to this:

```
deployments "nginx"
REVISION  CHANGE-CAUSE
1   <none>
```

- Get the rollout status to verify that the existing ReplicaSet has not changed:

```
kubectl get rs
```

The output is similar to this:

```
NAME               DESIRED   CURRENT   READY     AGE
nginx-2142116321   3         3         3         2m
```

- You can make as many updates as you wish, for example, update the resources that will be used:

```
kubectl set resources deployment/nginx-deployment -c=nginx --limits=cpu=200m,memory=512Mi
```

The output is similar to this:

```
deployment.apps/nginx-deployment resource requirements updated
```

The initial state of the Deployment prior to pausing its rollout will continue its function, but new updates to the Deployment will not have any effect as long as the Deployment rollout is paused.

- Eventually, resume the Deployment rollout and observe a new ReplicaSet coming up with all the new updates:

```
kubectl rollout resume deployment/nginx-deployment
```

The output is similar to this:

```
deployment.apps/nginx-deployment resumed
```

- Watch the status of the rollout until it's done.

```
kubectl get rs --watch
```

The output is similar to this:

```
NAME              DESIRED   CURRENT   READY     AGE
nginx-2142116321  2         2         2         2m
nginx-3926361531  2         2         0         6s
nginx-3926361531  2         2         1         18s
nginx-2142116321  1         2         2         2m
nginx-2142116321  1         2         2         2m
nginx-3926361531  3         2         1         18s
nginx-3926361531  3         2         1         18s
nginx-2142116321  1         1         1         2m
nginx-3926361531  3         3         1         18s
nginx-3926361531  3         3         2         19s
nginx-2142116321  0         1         1         2m
nginx-2142116321  0         1         1         2m
nginx-2142116321  0         0         0         2m
nginx-3926361531  3         3         3         20s
```

- Get the status of the latest rollout:

```
kubectl get rs
```

The output is similar to this:

```
NAME              DESIRED   CURRENT   READY     AGE
nginx-2142116321  0         0         0         2m
nginx-3926361531  3         3         3         28s
```

> **Note:**
> You cannot rollback a paused Deployment until you resume it.

Deployment status

A Deployment enters various states during its lifecycle. It can be progressing while rolling out a new ReplicaSet, it can be complete, or it can fail to progress.

Progressing Deployment

Kubernetes marks a Deployment as *progressing* when one of the following tasks is performed:

- The Deployment creates a new ReplicaSet.
- The Deployment is scaling up its newest ReplicaSet.
- The Deployment is scaling down its older ReplicaSet(s).
- New Pods become ready or available (ready for at least MinReadySeconds).

When the rollout becomes “progressing”, the Deployment controller adds a condition with the following attributes to the Deployment's `.status.conditions`：

- `type: Progressing`
- `status: "True"`
- `reason: NewReplicaSetCreated` | `reason: FoundNewReplicaSet` | `reason: ReplicaSetUpdated`

You can monitor the progress for a Deployment by using `kubectl rollout status`.

Complete Deployment

Kubernetes marks a Deployment as *complete* when it has the following characteristics:

- All of the replicas associated with the Deployment have been updated to the latest version you've specified, meaning any updates you've requested have been completed.
- All of the replicas associated with the Deployment are available.
- No old replicas for the Deployment are running.

When the rollout becomes “complete”, the Deployment controller sets a condition with the following attributes to the Deployment's `.status.conditions`：

- `type: Progressing`
- `status: "True"`
- `reason: NewReplicaSetAvailable`

This `Progressing` condition will retain a status value of `"True"` until a new rollout is initiated. The condition holds even when availability of replicas changes (which does instead affect the `Available` condition).

You can check if a Deployment has completed by using `kubectl rollout status`. If the rollout completed successfully, `kubectl rollout status` returns a zero exit code.

```
kubectl rollout status deployment/nginx-deployment
```

The output is similar to this:

```
Waiting for rollout to finish: 2 of 3 updated replicas are available...
deployment "nginx-deployment" successfully rolled out
```

and the exit status from `kubectl rollout` is 0 (success):

```
echo $?
```

```
0
```

Failed Deployment

Your Deployment may get stuck trying to deploy its newest ReplicaSet without ever completing. This can occur due to some of the following factors:

- Insufficient quota
- Readiness probe failures
- Image pull errors
- Insufficient permissions
- Limit ranges
- Application runtime misconfiguration

One way you can detect this condition is to specify a deadline parameter in your Deployment spec: (`.spec.progressDeadlineSeconds`). `.spec.progressDeadlineSeconds` denotes the number of seconds the Deployment controller waits before indicating (in the Deployment status) that the Deployment progress has stalled.

The following `kubectl` command sets the spec with `progressDeadlineSeconds` to make the controller report lack of progress of a rollout for a Deployment after 10 minutes:

```
kubectl patch deployment/nginx-deployment -p '{"spec":{"progressDeadlineSeconds":600}}'
```

The output is similar to this:

```
deployment.apps/nginx-deployment patched
```

Once the deadline has been exceeded, the Deployment controller adds a DeploymentCondition with the following attributes to the Deployment's `.status.conditions`:

- `type: Progressing`
- `status: "False"`
- `reason: ProgressDeadlineExceeded`

This condition can also fail early and is then set to status value of `"False"` due to reasons as `ReplicaSetCreateError`. Also, the deadline is not taken into account anymore once the Deployment rollout completes.

See the Kubernetes API conventions for more information on status conditions.

> **Note:**
> Kubernetes takes no action on a stalled Deployment other than to report a status condition with `reason: ProgressDeadlineExceeded`. Higher level orchestrators can take advantage of it and act accordingly, for example, rollback the Deployment to its previous version.

> **Note:**
> If you pause a Deployment rollout, Kubernetes does not check progress against your specified deadline. You can safely pause a Deployment rollout in the middle of a rollout and resume without triggering the condition for exceeding the deadline.

You may experience transient errors with your Deployments, either due to a low timeout that you have set or due to any other kind of error that can be treated as transient. For example, let's suppose you have insufficient quota. If you describe the Deployment you will notice the following section:

```
kubectl describe deployment nginx-deployment
```

The output is similar to this:

```
<...>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    ReplicaSetUpdated
  ReplicaFailure  True    FailedCreate
<...>
```

If you run `kubectl get deployment nginx-deployment -o yaml`, the Deployment status is similar to this:

```
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: 2016-10-04T12:25:39Z
    lastUpdateTime: 2016-10-04T12:25:39Z
    message: Replica set "nginx-deployment-4262182780" is progressing.
    reason: ReplicaSetUpdated
    status: "True"
    type: Progressing
  - lastTransitionTime: 2016-10-04T12:25:42Z
    lastUpdateTime: 2016-10-04T12:25:42Z
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: 2016-10-04T12:25:39Z
    lastUpdateTime: 2016-10-04T12:25:39Z
    message: 'Error creating: pods "nginx-deployment-4262182780-" is forbidden: exceeded quota:
      object-counts, requested: pods=1, used: pods=3, limited: pods=2'
    reason: FailedCreate
    status: "True"
    type: ReplicaFailure
  observedGeneration: 3
  replicas: 2
  unavailableReplicas: 2
```

Eventually, once the Deployment progress deadline is exceeded, Kubernetes updates the status and the reason for the Progressing condition:

```
Conditions:
  Type            Status  Reason
  ----            ------  ------
  Available       True    MinimumReplicasAvailable
  Progressing     False   ProgressDeadlineExceeded
  ReplicaFailure  True    FailedCreate
```

You can address an issue of insufficient quota by scaling down your Deployment, by scaling down other controllers you may be running, or by increasing quota in your namespace. If you satisfy the quota conditions and the Deployment controller then completes the Deployment rollout, you'll see the Deployment's status update with a successful condition (`status: "True"` and `reason: NewReplicaSetAvailable`).

```
Conditions:
  Type          Status  Reason
  ----          ------  ------
  Available     True    MinimumReplicasAvailable
  Progressing   True    NewReplicaSetAvailable
```

`type: Available` with `status: "True"` means that your Deployment has minimum availability. Minimum availability is dictated by the parameters specified in the deployment strategy. `type: Progressing` with `status: "True"` means that your Deployment is either in the middle of a rollout and it is progressing or that it has successfully completed its progress and the minimum required new replicas are available (see the Reason of the condition for the particulars - in our case `reason: NewReplicaSetAvailable` means that the Deployment is complete).

You can check if a Deployment has failed to progress by using `kubectl rollout status` . `kubectl rollout status` returns a non-zero exit code if the Deployment has exceeded the progression deadline.

```
kubectl rollout status deployment/nginx-deployment
```

The output is similar to this:

```
Waiting for rollout to finish: 2 out of 3 new replicas have been updated...
error: deployment "nginx" exceeded its progress deadline
```

and the exit status from `kubectl rollout` is 1 (indicating an error):

```
echo $?
```

```
1
```

Operating on a failed deployment

All actions that apply to a complete Deployment also apply to a failed Deployment. You can scale it up/down, roll back to a previous revision, or even pause it if you need to apply multiple tweaks in the Deployment Pod template.

Clean up Policy

You can set `.spec.revisionHistoryLimit` field in a Deployment to specify how many old ReplicaSets for this Deployment you want to retain. The rest will be garbage-collected in the background. By default, it is 10.

> **Note:**
> Explicitly setting this field to 0, will result in cleaning up all the history of your Deployment thus that Deployment will not be able to roll back.

The cleanup only starts **after** a Deployment reaches a complete state. If you set `.spec.revisionHistoryLimit` to 0, any rollout nonetheless triggers creation of a new ReplicaSet before Kubernetes removes the old one.

Even with a non-zero revision history limit, you can have more ReplicaSets than the limit you configure. For example, if pods are crash looping, and there are multiple rolling updates events triggered over time, you might end up with more ReplicaSets than the `.spec.revisionHistoryLimit` because the Deployment never reaches a complete state.

Canary Deployment

If you want to roll out releases to a subset of users or servers using the Deployment, you can create multiple Deployments, one for each release, following the canary pattern described in managing resources.

Writing a Deployment Spec

As with all other Kubernetes configs, a Deployment needs `.apiVersion`, `.kind`, and `.metadata` fields. For general information about working with config files, see deploying applications, configuring containers, and using kubectl to manage resources documents.

When the control plane creates new Pods for a Deployment, the `.metadata.name` of the Deployment is part of the basis for naming those Pods. The name of a Deployment must be a valid DNS subdomain value, but this can produce unexpected results for the Pod hostnames. For best compatibility, the name should follow the more restrictive rules for a DNS label.

A Deployment also needs a `.spec` section.

Pod Template

The `.spec.template` and `.spec.selector` are the only required fields of the `.spec`.

The `.spec.template` is a Pod template. It has exactly the same schema as a Pod, except it is nested and does not have an `apiVersion` or `kind`.

In addition to required fields for a Pod, a Pod template in a Deployment must specify appropriate labels and an appropriate restart policy. For labels, make sure not to overlap with other controllers. See selector.

Only a `.spec.template.spec.restartPolicy` equal to `Always` is allowed, which is the default if not specified.

Replicas

`.spec.replicas` is an optional field that specifies the number of desired Pods. It defaults to 1.

Should you manually scale a Deployment, example via `kubectl scale deployment deployment --replicas=X`, and then you update that Deployment based on a manifest (for example: by running `kubectl apply -f deployment.yaml`), then applying that manifest overwrites the manual scaling that you previously did.

If a HorizontalPodAutoscaler (or any similar API for horizontal scaling) is managing scaling for a Deployment, don't set `.spec.replicas`.

Instead, allow the Kubernetes control plane to manage the `.spec.replicas` field automatically.

Selector

`.spec.selector` is a required field that specifies a label selector for the Pods targeted by this Deployment.

`.spec.selector` must match `.spec.template.metadata.labels`, or it will be rejected by the API.

In API version `apps/v1`, `.spec.selector` and `.metadata.labels` do not default to `.spec.template.metadata.labels` if not set. So they must be set explicitly. Also note that `.spec.selector` is immutable after creation of the Deployment in `apps/v1`.

A Deployment may terminate Pods whose labels match the selector if their template is different from `.spec.template` or if the total number of such Pods exceeds `.spec.replicas`. It brings up new Pods with `.spec.template` if the number of Pods is less than the desired number.

> **Note:**
> You should not create other Pods whose labels match this selector, either directly, by creating another Deployment, or by creating another controller such as a ReplicaSet or a ReplicationController. If you do so, the first Deployment thinks that it created these other Pods. Kubernetes does not stop you from doing this.

If you have multiple controllers that have overlapping selectors, the controllers will fight with each other and won't behave correctly.

Strategy

`.spec.strategy` specifies the strategy used to replace old Pods by new ones. `.spec.strategy.type` can be "Recreate" or "RollingUpdate". "RollingUpdate" is the default value.

Recreate Deployment

All existing Pods are killed before new ones are created when `.spec.strategy.type==Recreate`.

> **Note:**
> This will only guarantee Pod termination previous to creation for upgrades. If you upgrade a Deployment, all Pods of the old revision will be terminated immediately. Successful removal is awaited before any Pod of the new revision is created. If you manually delete a Pod, the lifecycle is controlled by the ReplicaSet and the replacement will be created immediately (even if the old Pod is still in a Terminating state). If you need an "at most" guarantee for your Pods, you should consider using a StatefulSet.

Rolling Update Deployment

The Deployment updates Pods in a rolling update fashion (gradually scale down the old ReplicaSets and scale up the new one) when `.spec.strategy.type==RollingUpdate`. You can specify `maxUnavailable` and `maxSurge` to control the rolling update process.

Max Unavailable

`.spec.strategy.rollingUpdate.maxUnavailable` is an optional field that specifies the maximum number of Pods that can be unavailable during the update process. The value can be an absolute number (for example, 5) or a percentage of desired Pods (for example, 10%). The absolute number is calculated from percentage by rounding down. The value cannot be 0 if `.spec.strategy.rollingUpdate.maxSurge` is 0. The default value is 25%.

For example, when this value is set to 30%, the old ReplicaSet can be scaled down to 70% of desired Pods immediately when the rolling update starts. Once new Pods are ready, old ReplicaSet can be scaled down further, followed by scaling up the new ReplicaSet, ensuring that the total number of Pods available at all times during the update is at least 70% of the desired Pods.

Max Surge

`.spec.strategy.rollingUpdate.maxSurge` is an optional field that specifies the maximum number of Pods that can be created over the desired number of Pods. The value can be an absolute number (for example, 5) or a percentage of desired Pods (for example, 10%). The value cannot be 0 if `maxUnavailable` is 0. The absolute number is calculated from the percentage by rounding up. The default value is 25%.

For example, when this value is set to 30%, the new ReplicaSet can be scaled up immediately when the rolling update starts, such that the total number of old and new Pods does not exceed 130% of desired Pods. Once old Pods have been killed, the new ReplicaSet can be scaled up further, ensuring that the total number of Pods running at any time during the update is at most 130% of desired Pods.

Here are some Rolling Update Deployment examples that use the `maxUnavailable` and `maxSurge`:

Max Unavailable | Max Surge | Hybrid

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-deployment
 labels:
   app: nginx
spec:
 replicas: 3
 selector:
   matchLabels:
     app: nginx
 template:
   metadata:
     labels:
       app: nginx
   spec:
     containers:
     - name: nginx
       image: nginx:1.14.2
       ports:
       - containerPort: 80
 strategy:
   type: RollingUpdate
   rollingUpdate:
     maxUnavailable: 1
```

Progress Deadline Seconds

`.spec.progressDeadlineSeconds` is an optional field that specifies the number of seconds you want to wait for your Deployment to progress before the system reports back that the Deployment has failed progressing - surfaced as a condition with `type: Progressing`, `status: "False"`. and `reason: ProgressDeadlineExceeded` in the status of the resource. The Deployment controller will keep retrying the Deployment. This defaults to 600. In the future, once automatic rollback will be implemented, the Deployment controller will roll back a Deployment as soon as it observes such a condition.

If specified, this field needs to be greater than `.spec.minReadySeconds`.

Min Ready Seconds

`.spec.minReadySeconds` is an optional field that specifies the minimum number of seconds for which a newly created Pod should be ready without any of its containers crashing, for it to be considered available. This defaults to 0 (the Pod will be considered available as soon as it is ready). To learn more about when a Pod is considered ready, see Container Probes.

Terminating Pods

FEATURE STATE: Kubernetes v1.35 [beta](enabled by default)

You can see the terminating pods only if the `DeploymentReplicaSetTerminatingReplicas` feature gate is enabled on the API server and on the kube-controller-manager

Pods that become terminating due to deletion or scale down may take a long time to terminate, and may consume additional resources during that period. As a result, the total number of all pods can temporarily exceed `.spec.replicas`. Terminating pods can be tracked using the `.status.terminatingReplicas` field of the Deployment.

Revision History Limit

A Deployment's revision history is stored in the ReplicaSets it controls.

`.spec.revisionHistoryLimit` is an optional field that specifies the number of old ReplicaSets to retain to allow rollback. These old ReplicaSets consume resources in `etcd` and crowd the output of `kubectl get rs`. The configuration of each Deployment revision is stored in its ReplicaSets; therefore, once an old ReplicaSet is deleted, you lose the ability to rollback to that revision of Deployment. By default, 10 old ReplicaSets will be kept, however its ideal value depends on the frequency and stability of new Deployments.

More specifically, setting this field to zero means that all old ReplicaSets with 0 replicas will be cleaned up. In this case, a new Deployment rollout cannot be undone, since its revision history is cleaned up.

Paused

`.spec.paused` is an optional boolean field for pausing and resuming a Deployment. The only difference between a paused Deployment and one that is not paused, is that any changes into the PodTemplateSpec of the paused Deployment will not trigger new rollouts as long as it is paused. A Deployment is not paused by default when it is created.

What's next

- Learn more about Pods.
- Run a stateless application using a Deployment.
- Read the Deployment to understand the Deployment API.
- Read about PodDisruptionBudget and how you can use it to manage application availability during disruptions.
- Use kubectl to create a Deployment.

Feedback

Was this page helpful?

Yes No

Last modified March 15, 2026 at 3:21 PM PST: fix: replace deprecated argument `--cpu-percent` with `--cpu` (af93a0a732)

Service

Service

Expose an application running in your cluster behind a single outward-facing endpoint, even when the workload is split across multiple backends.

In Kubernetes, a Service is a method for exposing a network application that is running as one or more Pods in your cluster.

A key aim of Services in Kubernetes is that you don't need to modify your existing application to use an unfamiliar service discovery mechanism. You can run code in Pods, whether this is a code designed for a cloud-native world, or an older app you've containerized. You use a Service to make that set of Pods available on the network so that clients can interact with it.

If you use a Deployment to run your app, that Deployment can create and destroy Pods dynamically. From one moment to the next, you don't know how many of those Pods are working and healthy; you might not even know what those healthy Pods are named. Kubernetes Pods are created and destroyed to match the desired state of your cluster. Pods are ephemeral resources (you should not expect that an individual Pod is reliable and durable).

Each Pod gets its own IP address (Kubernetes expects network plugins to ensure this). For a given Deployment in your cluster, the set of Pods running in one moment in time could be different from the set of Pods running that application a moment later.

This leads to a problem: if some set of Pods (call them "backends") provides functionality to other Pods (call them "frontends") inside your cluster, how do the frontends find out and keep track of which IP address to connect to, so that the frontend can use the backend part of the workload?

Enter *Services*.

Services in Kubernetes

The Service API, part of Kubernetes, is an abstraction to help you expose groups of Pods over a network. Each Service object defines a logical set of endpoints (usually these endpoints are Pods) along with a policy about how to make those pods accessible.

For example, consider a stateless image-processing backend which is running with 3 replicas. Those replicas are fungible—frontends do not care which backend they use. While the actual Pods that compose the backend set may change, the frontend clients should not need to be aware of that, nor should they need to keep track of the set of backends themselves.

The Service abstraction enables this decoupling.

The set of Pods targeted by a Service is usually determined by a selector that you define. To learn about other ways to define Service endpoints, see Services *without* selectors.

If your workload speaks HTTP, you might choose to use an Ingress to control how web traffic reaches that workload. Ingress is not a Service type, but it acts as the entry point for your cluster. An Ingress lets you consolidate your routing rules into a single resource, so that you can expose multiple components of your workload, running separately in your cluster, behind a single listener.

The Gateway API for Kubernetes provides extra capabilities beyond Ingress and Service. You can add Gateway to your cluster - it is a family of extension APIs, implemented using CustomResourceDefinitions - and then use these to configure access to network services that are running in your cluster.

Cloud-native service discovery

If you're able to use Kubernetes APIs for service discovery in your application, you can query the API server for matching EndpointSlices. Kubernetes updates the EndpointSlices for a Service whenever the set of Pods in a Service changes.

For non-native applications, Kubernetes offers ways to place a network port or load balancer in between your application and the backend Pods.

Either way, your workload can use these service discovery mechanisms to find the target it wants to connect to.

Defining a Service

A Service is an object (the same way that a Pod or a ConfigMap is an object). You can create, view or modify Service definitions using the Kubernetes API. Usually you use a tool such as `kubectl` to make those API calls for you.

For example, suppose you have a set of Pods that each listen on TCP port 9376 and are labelled as `app.kubernetes.io/name=MyApp`. You can define a Service to publish that TCP listener:

service/simple-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Applying this manifest creates a new Service named "my-service" with the default ClusterIP service type. The Service targets TCP port 9376 on any Pod with the `app.kubernetes.io/name: MyApp` label.

Kubernetes assigns this Service an IP address (the *cluster IP*), that is used by the virtual IP address mechanism. For more details on that mechanism, read Virtual IPs and Service Proxies.

The controller for that Service continuously scans for Pods that match its selector, and then makes any necessary updates to the set of EndpointSlices for the Service.

The name of a Service object must be a valid RFC 1123 label name.

> **Note:**
> A Service can map *any* incoming `port` to a `targetPort`. By default and for convenience, the `targetPort` is set to the same value as the `port` field.

Port definitions

Port definitions in Pods have names, and you can reference these names in the `targetPort` attribute of a Service. For example, we can bind the `targetPort` of the Service to the Pod port in the following way:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app.kubernetes.io/name: proxy
  ports:
  - name: name-of-service-port
    protocol: TCP
    port: 80
    targetPort: http-web-svc

---
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app.kubernetes.io/name: proxy
spec:
  containers:
  - name: nginx
    image: nginx:stable
    ports:
      - containerPort: 80
        name: http-web-svc
```

This works even if there is a mixture of Pods in the Service using a single configured name, with the same network protocol available via different port numbers. This offers a lot of flexibility for deploying and evolving your Services. For example, you can change the port numbers that Pods expose in the next version of your backend software, without breaking clients.

The default protocol for Services is TCP; you can also use any other supported protocol.

Because many Services need to expose more than one port, Kubernetes supports multiple port definitions for a single Service. Each port definition can have the same `protocol`, or a different one.

Services without selectors

Services most commonly abstract access to Kubernetes Pods thanks to the selector, but when used with a corresponding set of EndpointSlices objects and without a selector, the Service can abstract other kinds of backends, including ones that run outside the cluster.

For example:

- You want to have an external database cluster in production, but in your test environment you use your own databases.
- You want to point your Service to a Service in a different Namespace or on another cluster.
- You are migrating a workload to Kubernetes. While evaluating the approach, you run only a portion of your backends in Kubernetes.

In any of these scenarios you can define a Service *without* specifying a selector to match Pods. For example:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
```

Because this Service has no selector, the corresponding EndpointSlice objects are not created automatically. You can map the Service to the network address and port where it's running, by adding an EndpointSlice object manually. For example:

```yaml
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: my-service-1 # by convention, use the name of the Service
                     # as a prefix for the name of the EndpointSlice
  labels:
    # You should set the "kubernetes.io/service-name" label.
    # Set its value to match the name of the Service
    kubernetes.io/service-name: my-service
addressType: IPv4
ports:
  - name: http # should match with the name of the service port defined above
    appProtocol: http
    protocol: TCP
    port: 9376
endpoints:
  - addresses:
      - "10.4.5.6"
  - addresses:
      - "10.1.2.3"
```

Custom EndpointSlices

When you create an EndpointSlice object for a Service, you can use any name for the EndpointSlice. Each EndpointSlice in a namespace must have a unique name. You link an EndpointSlice to a Service by setting the `kubernetes.io/service-name` label on that EndpointSlice.

> **Note:**
> The endpoint IPs *must not* be: loopback (127.0.0.0/8 for IPv4, ::1/128 for IPv6), or link-local (169.254.0.0/16 and 224.0.0.0/24 for IPv4, fe80::/64 for IPv6).
>
> The endpoint IP addresses cannot be the cluster IPs of other Kubernetes Services, because kube-proxy doesn't support virtual IPs as a destination.

For an EndpointSlice that you create yourself, or in your own code, you should also pick a value to use for the label `endpointslice.kubernetes.io/managed-by`. If you create your own controller code to manage EndpointSlices, consider using a value similar to `"my-domain.example/name-of-controller"`. If you are using a third party tool, use the name of the tool in all-lowercase and change spaces and other punctuation to dashes (`-`). If people are directly using a tool such as `kubectl` to manage EndpointSlices, use a name that describes this manual management, such as `"staff"` or `"cluster-admins"`. You should avoid using the reserved value `"controller"`, which identifies EndpointSlices managed by Kubernetes' own control plane.

Accessing a Service without a selector

Accessing a Service without a selector works the same as if it had a selector. In the example for a Service without a selector, traffic is routed to one of the two endpoints defined in the EndpointSlice manifest: a TCP connection to 10.1.2.3 or 10.4.5.6, on port 9376.

> **Note:**
> The Kubernetes API server does not allow proxying to endpoints that are not mapped to pods. Actions such as `kubectl port-forward service/<service-name> forwardedPort:servicePort` where the service has no selector will fail due to this constraint. This prevents the Kubernetes API server from being used as a proxy to endpoints the caller may not be authorized to access.

An `ExternalName` Service is a special case of Service that does not have selectors and uses DNS names instead. For more information, see the ExternalName section.

EndpointSlices

FEATURE STATE: Kubernetes v1.21 [stable]

EndpointSlices are objects that represent a subset (a *slice*) of the backing network endpoints for a Service.

Your Kubernetes cluster tracks how many endpoints each EndpointSlice represents. If there are so many endpoints for a Service that a threshold is reached, then Kubernetes adds another empty EndpointSlice and stores new endpoint information there. By default, Kubernetes makes a new EndpointSlice once the existing EndpointSlices all contain at least 100 endpoints. Kubernetes does not make the new EndpointSlice until an extra endpoint needs to be added.

See EndpointSlices for more information about this API.

Endpoints (deprecated)

FEATURE STATE: Kubernetes v1.33 [deprecated]

The EndpointSlice API is the evolution of the older Endpoints API. The deprecated Endpoints API has several problems relative to EndpointSlice:

- It does not support dual-stack clusters.
- It does not contain information needed to support newer features, such as trafficDistribution.
- It will truncate the list of endpoints if it is too long to fit in a single object.

Because of this, it is recommended that all clients use the EndpointSlice API rather than Endpoints.

Over-capacity endpoints

Kubernetes limits the number of endpoints that can fit in a single Endpoints object. When there are over 1000 backing endpoints for a Service, Kubernetes truncates the data in the Endpoints object. Because a Service can be linked with more than one EndpointSlice, the 1000 backing endpoint limit only affects the legacy Endpoints API.

In that case, Kubernetes selects at most 1000 possible backend endpoints to store into the Endpoints object, and sets an annotation on the Endpoints: `endpoints.kubernetes.io/over-capacity: truncated` . The control plane also removes that annotation if the number of backend Pods drops below 1000.

Traffic is still sent to backends, but any load balancing mechanism that relies on the legacy Endpoints API only sends traffic to at most 1000 of the available backing endpoints.

The same API limit means that you cannot manually update an Endpoints to have more than 1000 endpoints.

Application protocol

FEATURE STATE: Kubernetes v1.20 [stable]

The `appProtocol` field provides a way to specify an application protocol for each Service port. This is used as a hint for implementations to offer richer behavior for protocols that they understand. The value of this field is mirrored by the corresponding Endpoints and EndpointSlice objects.

This field follows standard Kubernetes label syntax. Valid values are one of:

- IANA standard service names.
- Implementation-defined prefixed names such as `mycompany.com/my-custom-protocol` .
- Kubernetes-defined prefixed names:

Protocol	Description
`kubernetes.io/h2c`	HTTP/2 over cleartext as described in RFC 7540

Protocol	Description
`kubernetes.io/ws`	WebSocket over cleartext as described in RFC 6455
`kubernetes.io/wss`	WebSocket over TLS as described in RFC 6455

Multi-port Services

For some Services, you need to expose more than one port. Kubernetes lets you configure multiple port definitions on a Service object. When using multiple ports for a Service, you must give all of your ports names so that these are unambiguous. For example:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
    - name: https
      protocol: TCP
      port: 443
      targetPort: 9377
```

> **Note:**
> As with Kubernetes names in general, names for ports must only contain lowercase alphanumeric characters and `-`. Port names must also start and end with an alphanumeric character.
>
> For example, the names `123-abc` and `web` are valid, but `123_abc` and `-web` are not.

Service type

For some parts of your application (for example, frontends) you may want to expose a Service onto an external IP address, one that's accessible from outside of your cluster.

Kubernetes Service types allow you to specify what kind of Service you want.

The available `type` values and their behaviors are:

`ClusterIP`

Exposes the Service on a cluster-internal IP. Choosing this value makes the Service only reachable from within the cluster. This is the default that is used if you don't explicitly specify a `type` for a Service. You can expose the Service to the public internet using an Ingress or a Gateway.

`NodePort`

Exposes the Service on each Node's IP at a static port (the `NodePort`). To make the node port available, Kubernetes sets up a cluster IP address, the same as if you had requested a Service of `type: ClusterIP`.

`LoadBalancer`

Exposes the Service externally using an external load balancer. Kubernetes does not directly offer a load balancing component; you must provide one, or you can integrate your Kubernetes cluster with a cloud provider.

`ExternalName`

Maps the Service to the contents of the `externalName` field (for example, to the hostname `api.foo.bar.example`). The mapping configures your cluster's DNS server to return a `CNAME` record with that external hostname value. No proxying of any kind is set up.

The `type` field in the Service API is designed as nested functionality - each level adds to the previous. However there is an exception to this nested design. You can define a `LoadBalancer` Service by disabling the load balancer `NodePort` allocation.

`type: ClusterIP`

This default Service type assigns an IP address from a pool of IP addresses that your cluster has reserved for that purpose.

Several of the other types for Service build on the `ClusterIP` type as a foundation.

If you define a Service that has the `.spec.clusterIP` set to `"None"` then Kubernetes does not assign an IP address. See headless Services for more information.

Choosing your own IP address

You can specify your own cluster IP address as part of a `Service` creation request. To do this, set the `.spec.clusterIP` field. For example, if you already have an existing DNS entry that you wish to reuse, or legacy systems that are configured for a specific IP address and difficult to re-configure.

The IP address that you choose must be a valid IPv4 or IPv6 address from within the `service-cluster-ip-range` CIDR range that is configured for the API server. If you try to create a Service with an invalid `clusterIP` address value, the API server will return a 422 HTTP status code to indicate that there's a problem.

Read avoiding collisions to learn how Kubernetes helps reduce the risk and impact of two different Services both trying to use the same IP address.

`type: NodePort`

If you set the `type` field to `NodePort`, the Kubernetes control plane allocates a port from a range specified by `--service-node-port-range` flag (default: 30000-32767). Each node proxies that port (the same port number on every Node) into your Service. Your Service reports the allocated port in its `.spec.ports[*].nodePort` field.

Using a NodePort gives you the freedom to set up your own load balancing solution, to configure environments that are not fully supported by Kubernetes, or even to expose one or more nodes' IP addresses directly.

For a node port Service, Kubernetes additionally allocates a port (TCP, UDP or SCTP to match the protocol of the Service). Every node in the cluster configures itself to listen on that assigned port and to forward traffic to one of the ready endpoints associated with that Service. You'll be able to contact the `type: NodePort` Service, from outside the cluster, by connecting to any node using the appropriate protocol (for example: TCP), and the appropriate port (as assigned to that Service).

Choosing your own port

If you want a specific port number, you can specify a value in the `nodePort` field. The control plane will either allocate you that port or report that the API transaction failed. This means that you need to take care of possible port collisions yourself. You also have to use a valid port number, one that's inside the range configured for NodePort use.

Here is an example manifest for a Service of `type: NodePort` that specifies a NodePort value (30007, in this example):

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - port: 80
      # By default and for convenience, the `targetPort` is set to
      # the same value as the `port` field.
      targetPort: 80
      # Optional field
      # By default and for convenience, the Kubernetes control plane
      # will allocate a port from a range (default: 30000-32767)
      nodePort: 30007
```

Reserve Nodeport ranges to avoid collisions

The policy for assigning ports to NodePort services applies to both the auto-assignment and the manual assignment scenarios. When a user wants to create a NodePort service that uses a specific port, the target port may conflict with another port that has already been assigned.

To avoid this problem, the port range for NodePort services is divided into two bands. Dynamic port assignment uses the upper band by default, and it may use the lower band once the upper band has been exhausted. Users can then allocate from the lower band with a lower risk of port collision.

When using the default NodePort range 30000-32767, the bands are partitioned as follows:

- Static band: 30000-30085
- Dynamic band: 30086-32767

See Avoid Collisions Assigning Ports to NodePort Services for more details on how the static and dynamic bands are calculated.

Custom IP address configuration for `type: NodePort` Services

You can set up nodes in your cluster to use a particular IP address for serving node port services. You might want to do this if each node is connected to multiple networks (for example: one network for application traffic, and another network for traffic between nodes and the control plane).

If you want to specify particular IP address(es) to proxy the port, you can set the `--nodeport-addresses` flag for kube-proxy or the equivalent `nodePortAddresses` field of the kube-proxy configuration file to particular IP block(s).

This flag takes a comma-delimited list of IP blocks (e.g. `10.0.0.0/8`, `192.0.2.0/25`) to specify IP address ranges that kube-proxy should consider as local to this node.

For example, if you start kube-proxy with the `--nodeport-addresses=127.0.0.0/8` flag, kube-proxy only selects the loopback interface for NodePort Services. The default for `--nodeport-addresses` is an empty list. This means that kube-proxy should consider all available network interfaces for NodePort. (That's also compatible with earlier Kubernetes releases.)

> **Note:**
> This Service is visible as `<NodeIP>:spec.ports[*].nodePort` and `.spec.clusterIP:spec.ports[*].port`. If the `--nodeport-addresses` flag for kube-proxy or the equivalent field in the kube-proxy configuration file is set, `<NodeIP>` would be a filtered node IP address (or possibly IP addresses).

`type: LoadBalancer`

On cloud providers which support external load balancers, setting the `type` field to `LoadBalancer` provisions a load balancer for your Service. The actual creation of the load balancer happens asynchronously, and information about the provisioned balancer is published in the Service's `.status.loadBalancer` field. For example:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  clusterIP: 10.0.171.239
  type: LoadBalancer
status:
  loadBalancer:
    ingress:
    - ip: 192.0.2.127
```

Traffic from the external load balancer is directed at the backend Pods. The cloud provider decides how it is load balanced.

To implement a Service of `type: LoadBalancer`, Kubernetes typically starts off by making the changes that are equivalent to you requesting a Service of `type: NodePort`. The cloud-controller-manager component then configures the external load balancer to forward traffic to that assigned node port.

You can configure a load balanced Service to omit assigning a node port, provided that the cloud provider implementation supports this.

Some cloud providers allow you to specify the `loadBalancerIP`. In those cases, the load-balancer is created with the user-specified `loadBalancerIP`. If the `loadBalancerIP` field is not specified, the load balancer is set up with an ephemeral IP address. If you specify a `loadBalancerIP` but your cloud provider does not support the feature, the `loadbalancerIP` field that you set is ignored.

> **Note:**
> The `.spec.loadBalancerIP` field for a Service was deprecated in Kubernetes v1.24.
>
> This field was under-specified and its meaning varies across implementations. It also cannot support dual-stack networking. This field may be removed in a future API version.
>
> If you're integrating with a provider that supports specifying the load balancer IP address(es) for a Service via a (provider specific) annotation, you should switch to doing that.
>
> If you are writing code for a load balancer integration with Kubernetes, avoid using this field. You can integrate with Gateway rather than Service, or you can define your own (provider specific) annotations on the Service that specify the equivalent detail.

Node liveness impact on load balancer traffic

Load balancer health checks are critical to modern applications. They are used to determine which server (virtual machine, or IP address) the load balancer should dispatch traffic to. The Kubernetes APIs do not define how health checks have to be implemented for Kubernetes managed load balancers, instead it's the cloud providers (and the people implementing integration code) who decide on the behavior. Load balancer health checks are extensively used within the context of supporting the `externalTrafficPolicy` field for Services.

Load balancers with mixed protocol types

FEATURE STATE: Kubernetes v1.26 [stable](enabled by default)

By default, for LoadBalancer type of Services, when there is more than one port defined, all ports must have the same protocol, and the protocol must be one which is supported by the cloud provider.

The feature gate `MixedProtocolLBService` (enabled by default for the kube-apiserver as of v1.24) allows the use of different protocols for LoadBalancer type of Services, when there is more than one port defined.

> **Note:**

> The set of protocols that can be used for load balanced Services is defined by your cloud provider; they may impose restrictions beyond what the Kubernetes API enforces.

Disabling load balancer NodePort allocation

ⓘ **FEATURE STATE:** Kubernetes v1.24 [stable]

You can optionally disable node port allocation for a Service of `type: LoadBalancer`, by setting the field `spec.allocateLoadBalancerNodePorts` to `false`. This should only be used for load balancer implementations that route traffic directly to pods as opposed to using node ports. By default, `spec.allocateLoadBalancerNodePorts` is `true` and type LoadBalancer Services will continue to allocate node ports. If `spec.allocateLoadBalancerNodePorts` is set to `false` on an existing Service with allocated node ports, those node ports will **not** be de-allocated automatically. You must explicitly remove the `nodePorts` entry in every Service port to de-allocate those node ports.

Specifying class of load balancer implementation

ⓘ **FEATURE STATE:** Kubernetes v1.24 [stable]

For a Service with `type` set to `LoadBalancer`, the `.spec.loadBalancerClass` field enables you to use a load balancer implementation other than the cloud provider default.

By default, `.spec.loadBalancerClass` is not set and a `LoadBalancer` type of Service uses the cloud provider's default load balancer implementation if the cluster is configured with a cloud provider using the `--cloud-provider` component flag.

If you specify `.spec.loadBalancerClass`, it is assumed that a load balancer implementation that matches the specified class is watching for Services. Any default load balancer implementation (for example, the one provided by the cloud provider) will ignore Services that have this field set. `spec.loadBalancerClass` can be set on a Service of type `LoadBalancer` only. Once set, it cannot be changed. The value of `spec.loadBalancerClass` must be a label-style identifier, with an optional prefix such as "`internal-vip`" or "`example.com/internal-vip`". Unprefixed names are reserved for end-users.

Load balancer IP address mode

For a Service of `type: LoadBalancer`, a controller can set `.status.loadBalancer.ingress.ipMode`. The `.status.loadBalancer.ingress.ipMode` specifies how the load-balancer IP behaves. It may be specified only when the `.status.loadBalancer.ingress.ip` field is also specified.

There are two possible values for `.status.loadBalancer.ingress.ipMode`: "VIP" and "Proxy". The default value is "VIP" meaning that traffic is delivered to the node with the destination set to the load-balancer's IP and port. There are two cases when setting this to "Proxy", depending on how the load-balancer from the cloud provider delivers the traffics:

- If the traffic is delivered to the node then DNATed to the pod, the destination would be set to the node's IP and node port;
- If the traffic is delivered directly to the pod, the destination would be set to the pod's IP and port.

Service implementations may use this information to adjust traffic routing.

Internal load balancer

In a mixed environment it is sometimes necessary to route traffic from Services inside the same (virtual) network address block.

In a split-horizon DNS environment you would need two Services to be able to route both external and internal traffic to your endpoints.

To set an internal load balancer, add one of the following annotations to your Service depending on the cloud service provider you're using:

Default | GCP | AWS | Azure | IBM Cloud | OpenStack | Baidu Cloud | Tencent Cloud | Alibaba Cloud | OCI

Select one of the tabs.

`type: ExternalName`

Services of type ExternalName map a Service to a DNS name, not to a typical selector such as `my-service` or `cassandra`. You specify these Services with the `spec.externalName` parameter.

This Service definition, for example, maps the `my-service` Service in the `prod` namespace to `my.database.example.com`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

> **Note:**
> A Service of `type: ExternalName` accepts an IPv4 address string, but treats that string as a DNS name comprised of digits, not as an IP address (the internet does not however allow such names in DNS). Services with external names that resemble IPv4 addresses are not resolved by DNS servers.
>
> If you want to map a Service directly to a specific IP address, consider using headless Services.

When looking up the host `my-service.prod.svc.cluster.local`, the cluster DNS Service returns a `CNAME` record with the value `my.database.example.com`. Accessing `my-service` works in the same way as other Services but with the crucial difference that redirection happens at the DNS level rather than via proxying or forwarding. Should you later decide to move your database into your cluster, you can start its Pods, add appropriate selectors or endpoints, and change the Service's `type`.

> **Caution:**
> You may have trouble using ExternalName for some common protocols, including HTTP and HTTPS. If you use ExternalName then the hostname used by clients inside your cluster is different from the name that the ExternalName references.
>
> For protocols that use hostnames this difference may lead to errors or unexpected responses. HTTP requests will have a `Host:` header that the origin server does not recognize; TLS servers will not be able to provide a certificate matching the hostname that the client connected to.

Headless Services

Sometimes you don't need load-balancing and a single Service IP. In this case, you can create what are termed *headless Services*, by explicitly specifying `"None"` for the cluster IP address (`.spec.clusterIP`).

You can use a headless Service to interface with other service discovery mechanisms, without being tied to Kubernetes' implementation.

For headless Services, a cluster IP is not allocated, kube-proxy does not handle these Services, and there is no load balancing or proxying done by the platform for them.

A headless Service allows a client to connect to whichever Pod it prefers, directly. Services that are headless don't configure routes and packet forwarding using virtual IP addresses and proxies; instead, headless Services report the endpoint IP addresses of the individual pods via internal DNS records, served through the cluster's DNS service. To define a headless Service, you make a Service with `.spec.type` set to ClusterIP (which is also the default for `type`), and you additionally set `.spec.clusterIP` to None.

The string value None is a special case and is not the same as leaving the `.spec.clusterIP` field unset.

How DNS is automatically configured depends on whether the Service has selectors defined:

With selectors

For headless Services that define selectors, the endpoints controller creates EndpointSlices in the Kubernetes API, and modifies the DNS configuration to return A or AAAA records (IPv4 or IPv6 addresses) that point directly to the Pods backing the Service.

Without selectors

For headless Services that do not define selectors, the control plane does not create EndpointSlice objects. However, the DNS system looks for and configures either:

- DNS CNAME records for `type: ExternalName` Services.
- DNS A / AAAA records for all IP addresses of the Service's ready endpoints, for all Service types other than `ExternalName`.
  - For IPv4 endpoints, the DNS system creates A records.
  - For IPv6 endpoints, the DNS system creates AAAA records.

When you define a headless Service without a selector, the `port` must match the `targetPort`.

Discovering services

For clients running inside your cluster, Kubernetes supports two primary modes of finding a Service: environment variables and DNS.

Environment variables

When a Pod is run on a Node, the kubelet adds a set of environment variables for each active Service. It adds `{SVCNAME}_SERVICE_HOST` and `{SVCNAME}_SERVICE_PORT` variables, where the Service name is upper-cased and dashes are converted to underscores.

For example, the Service `redis-primary` which exposes TCP port 6379 and has been allocated cluster IP address 10.0.0.11, produces the following environment variables:

```
REDIS_PRIMARY_SERVICE_HOST=10.0.0.11
REDIS_PRIMARY_SERVICE_PORT=6379
REDIS_PRIMARY_PORT=tcp://10.0.0.11:6379
REDIS_PRIMARY_PORT_6379_TCP=tcp://10.0.0.11:6379
REDIS_PRIMARY_PORT_6379_TCP_PROTO=tcp
REDIS_PRIMARY_PORT_6379_TCP_PORT=6379
REDIS_PRIMARY_PORT_6379_TCP_ADDR=10.0.0.11
```

> **Note:**
> When you have a Pod that needs to access a Service, and you are using the environment variable method to publish the port and cluster IP to the client Pods, you must create the Service *before* the client Pods come into existence. Otherwise, those client Pods won't have their environment variables populated.
>
> If you only use DNS to discover the cluster IP for a Service, you don't need to worry about this ordering issue.

Kubernetes also supports and provides variables that are compatible with Docker Engine's "*legacy container links*" feature. You can read `makeLinkVariables` to see how this is implemented in Kubernetes.

DNS

You can (and almost always should) set up a DNS service for your Kubernetes cluster using an add-on.

A cluster-aware DNS server, such as CoreDNS, watches the Kubernetes API for new Services and creates a set of DNS records for each one. If DNS has been enabled throughout your cluster then all Pods should automatically be able to resolve Services by their DNS name.

For example, if you have a Service called `my-service` in a Kubernetes namespace `my-ns`, the control plane and the DNS Service acting together create a DNS record for `my-service.my-ns`. Pods in the `my-ns` namespace should be able to find the service by doing a name lookup for `my-service` (`my-service.my-ns` would also work).

Pods in other namespaces must qualify the name as `my-service.my-ns`. These names will resolve to the cluster IP assigned for the Service.

Kubernetes also supports DNS SRV (Service) records for named ports. If the `my-service.my-ns` Service has a port named `http` with the protocol set to `TCP`, you can do a DNS SRV query for `_http._tcp.my-service.my-ns` to discover the port number for `http`, as well as the IP address.

The Kubernetes DNS server is the only way to access `ExternalName` Services. You can find more information about `ExternalName` resolution in DNS for Services and Pods.

Virtual IP addressing mechanism

Read Virtual IPs and Service Proxies explains the mechanism Kubernetes provides to expose a Service with a virtual IP address.

Traffic policies

You can set the `.spec.internalTrafficPolicy` and `.spec.externalTrafficPolicy` fields to control how Kubernetes routes traffic to healthy ("ready") backends.

See Traffic Policies for more details.

Traffic distribution control

The `.spec.trafficDistribution` field provides another way to influence traffic routing within a Kubernetes Service. While traffic policies focus on strict semantic guarantees, traffic distribution allows you to express *preferences* (such as routing to topologically closer endpoints). This can help optimize for performance, cost, or reliability. In Kubernetes 1.36, the following values are supported:

`PreferSameZone`

Indicates a preference for routing traffic to endpoints that are in the same zone as the client.

`PreferSameNode`

Indicates a preference for routing traffic to endpoints that are on the same node as the client.

`PreferClose` (deprecated)

This is an older alias for `PreferSameZone` that is less clear about the semantics.

If the field is not set, the implementation will apply its default routing strategy.

See Traffic Distribution for more details

Session stickiness

If you want to make sure that connections from a particular client are passed to the same Pod each time, you can configure session affinity based on the client's IP address. Read session affinity to learn more.

External IPs

FEATURE STATE: Kubernetes v1.36 [deprecated]

All users should begin migrating away from `externalIPs`. Consider using an external load balancer controller or a Gateway API implementation instead.

If there are external IPs that route to one or more cluster nodes, Kubernetes Services can be exposed on those `externalIPs`. When network traffic arrives into the cluster, with the external IP (as destination IP) and the port matching that Service, rules and routes that Kubernetes has configured ensure that the traffic is routed to one of the endpoints for that Service.

When you define a Service, you can specify `externalIPs` for any service type. In the example below, the Service named `"my-service"` can be accessed by clients using TCP, on `"198.51.100.32:80"` (calculated from `.spec.externalIPs[]` and `.spec.ports[].port`).

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 49152
  externalIPs:
    - 198.51.100.32
```

> **Note:**
> Kubernetes does not manage allocation of `externalIPs`; these are the responsibility of the cluster administrator.

API Object

Service is a top-level resource in the Kubernetes REST API. You can find more details about the Service API object.

What's next

Learn more about Services and how they fit into Kubernetes:

- Follow the Connecting Applications with Services tutorial.
- Read about Ingress, which exposes HTTP and HTTPS routes from outside the cluster to Services within your cluster.
- Read about Gateway, an extension to Kubernetes that provides more flexibility than Ingress.

For more context, read the following:

- Virtual IPs and Service Proxies
- EndpointSlices
- Service API reference
- EndpointSlice API reference
- Endpoint API reference (legacy)

Feedback

Was this page helpful?

Yes No

Last modified April 22, 2026 at 7:24 PM PST: Use feature-state shortcode for externalIPs deprecation (434ae53916)

Autoscaling Workloads

Autoscaling Workloads

With autoscaling, you can automatically update your workloads in one way or another. This allows your cluster to react to changes in resource demand more elastically and efficiently.

In Kubernetes, you can *scale* a workload depending on the current demand of resources. This allows your cluster to react to changes in resource demand more elastically and efficiently.

When you scale a workload, you can either increase or decrease the number of replicas managed by the workload, or adjust the resources available to the replicas in-place.

The first approach is referred to as *horizontal scaling*, while the second is referred to as *vertical scaling*.

There are manual and automatic ways to scale your workloads, depending on your use case.

Scaling workloads manually

Kubernetes supports *manual scaling* of workloads. Horizontal scaling can be done using the `kubectl` CLI. For vertical scaling, you need to *patch* the resource definition of your workload.

See below for examples of both strategies.

- **Horizontal scaling**: Running multiple instances of your app
- **Vertical scaling**: Resizing CPU and memory resources assigned to containers

Scaling workloads automatically

Kubernetes also supports *automatic scaling* of workloads, which is the focus of this page.

The concept of *Autoscaling* in Kubernetes refers to the ability to automatically update an object that manages a set of Pods (for example a Deployment).

Scaling workloads horizontally

In Kubernetes, you can automatically scale a workload horizontally using a HorizontalPodAutoscaler (HPA).

It is implemented as a Kubernetes API resource and a controller and periodically adjusts the number of replicas in a workload to match observed resource utilization such as CPU or memory usage.

There is a walkthrough tutorial of configuring a HorizontalPodAutoscaler for a Deployment.

Scaling workloads vertically

ⓘ **FEATURE STATE:** Kubernetes v1.25 [stable]

You can automatically scale a workload vertically using a VerticalPodAutoscaler (VPA). Unlike the HPA, the VPA doesn't come with Kubernetes by default, but is a an add-on that you or a cluster administrator may need to deploy before you can use it.

Once installed, it allows you to create CustomResourceDefinitions (CRDs) for your workloads which define *how* and *when* to scale the resources of the managed replicas.

> **Note:**
> You will need to have the Metrics Server installed to your cluster for the VPA to work.

In-place pod vertical scaling

FEATURE STATE: Kubernetes v1.35 [stable](enabled by default)

As of Kubernetes 1.36, VPA does not support resizing pods in-place, but this integration is being worked on. For manually resizing pods in-place, see Resize Container Resources In-Place.

Autoscaling based on cluster size

For workloads that need to be scaled based on the size of the cluster (for example `cluster-dns` or other system components), you can use the *Cluster Proportional Autoscaler*. Just like the VPA, it is not part of the Kubernetes core, but hosted as its own project on GitHub.

The Cluster Proportional Autoscaler watches the number of schedulable nodes and cores and scales the number of replicas of the target workload accordingly.

If the number of replicas should stay the same, you can scale your workloads vertically according to the cluster size using the *Cluster Proportional Vertical Autoscaler*. The project is **currently in beta** and can be found on GitHub.

While the Cluster Proportional Autoscaler scales the number of replicas of a workload, the Cluster Proportional Vertical Autoscaler adjusts the resource requests for a workload (for example a Deployment or DaemonSet) based on the number of nodes and/or cores in the cluster.

Event driven Autoscaling

It is also possible to scale workloads based on events, for example using the *Kubernetes Event Driven Autoscaler* (**KEDA**).

KEDA is a CNCF-graduated project enabling you to scale your workloads based on the number of events to be processed, for example the amount of messages in a queue. There exists a wide range of adapters for different event sources to choose from.

Autoscaling based on schedules

Another strategy for scaling your workloads is to **schedule** the scaling operations, for example in order to reduce resource consumption during off-peak hours.

Similar to event driven autoscaling, such behavior can be achieved using KEDA in conjunction with its `Cron` scaler. The `Cron` scaler allows you to define schedules (and time zones) for scaling your workloads in or out.

Scaling cluster infrastructure

If scaling workloads isn't enough to meet your needs, you can also scale your cluster infrastructure itself.

Scaling the cluster infrastructure normally means adding or removing nodes. Read Node autoscaling for more information.

What's next

- Learn more about scaling horizontally
  - Scale a StatefulSet
  - HorizontalPodAutoscaler Walkthrough
- Resize Container Resources In-Place
- Autoscale the DNS Service in a Cluster
- Learn about Node autoscaling

Feedback

Was this page helpful?

Yes No

Last modified November 23, 2025 at 1:47 PM PST: Move HorizontalPodAutoscaler concept page (57e1fdd7f9)

Horizontal Pod Autoscaling

Horizontal Pod Autoscaling

In Kubernetes, a *HorizontalPodAutoscaler* automatically updates a workload resource (such as a Deployment or StatefulSet), with the aim of automatically scaling capacity to match demand.

Horizontal scaling means that the response to increased load is to deploy more Pods. This is different from *vertical* scaling, which for Kubernetes would mean assigning more resources (for example: memory or CPU) to the Pods that are already running for the workload.

If the load decreases, and the number of Pods is above the configured minimum, the HorizontalPodAutoscaler instructs the workload resource (the Deployment, StatefulSet, or other similar resource) to scale back down.

Horizontal pod autoscaling does not apply to objects that can't be scaled (for example: a DaemonSet.)

The HorizontalPodAutoscaler is implemented as a Kubernetes API resource and a controller. The resource determines the behavior of the controller. The horizontal pod autoscaling controller, running within the Kubernetes control plane, periodically adjusts the desired scale of its target (for example, a Deployment) to match observed metrics such as average CPU utilization, average memory utilization, or any other custom metric you specify.

There is walkthrough example of using horizontal pod autoscaling.

How does a HorizontalPodAutoscaler work?

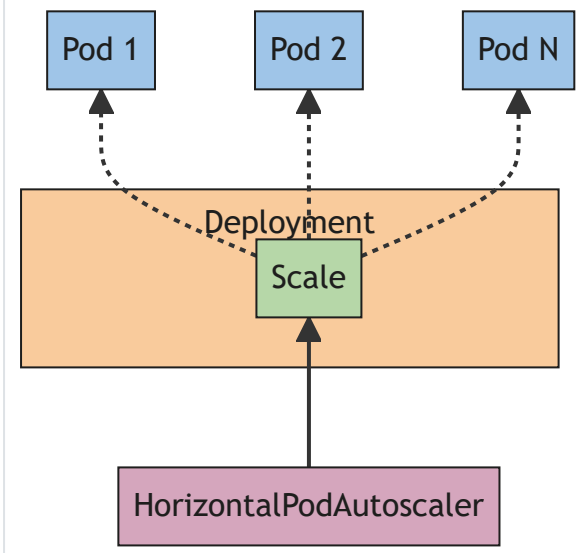


Figure 1. HorizontalPodAutoscaler controls the scale of a Deployment and its ReplicaSet

Kubernetes implements horizontal pod autoscaling as a control loop that runs intermittently (it is not a continuous process). The interval is set by the `--horizontal-pod-autoscaler-sync-period` parameter to the `kube-controller-manager` (and the default interval is 15 seconds).

Once during each period, the controller manager queries the resource utilization against the metrics specified in each HorizontalPodAutoscaler definition. The controller manager finds the target resource defined by the `scaleTargetRef`, then selects the pods based on the target resource's `.spec.selector` labels, and obtains the metrics from either the resource metrics API (for per-pod resource metrics), or the custom metrics API (for all other metrics).

- For per-pod resource metrics (like CPU), the controller fetches the metrics from the resource metrics API for each Pod targeted by the HorizontalPodAutoscaler. Then, if a target utilization value is set, the controller calculates the utilization value as a percentage of the equivalent resource request on the containers in each Pod. If a target raw value is set, the raw metric values are used directly. The controller then takes the mean of the utilization or the raw value (depending on the type of target specified) across all targeted Pods, and produces a ratio used to scale the number of desired replicas.

  Please note that if some of the Pod's containers do not have the relevant resource request set, CPU utilization for the Pod will not be defined and the autoscaler will not take any action for that metric. See the algorithm details section below for more information about how the autoscaling algorithm works.

- For per-pod custom metrics, the controller functions similarly to per-pod resource metrics, except that it works with raw values, not utilization values.
- For object metrics and external metrics, a single metric is fetched, which describes the object in question. This metric is compared to the target value, to produce a ratio as above. In the `autoscaling/v2` API version, this value can optionally be divided by the number of Pods before the comparison is made.

The common use for HorizontalPodAutoscaler is to configure it to fetch metrics from aggregated APIs (`metrics.k8s.io` , `custom.metrics.k8s.io` , or `external.metrics.k8s.io`). The `metrics.k8s.io` API is usually provided by an add-on named Metrics Server, which needs to be launched separately. For more information about resource metrics, see Metrics Server.

Support for metrics APIs explains the stability guarantees and support status for these different APIs.

The HorizontalPodAutoscaler controller accesses corresponding workload resources that support scaling (such as Deployments and StatefulSet). These resources each have a subresource named `scale` , an interface that allows you to dynamically set the number of replicas and examine each of their current states. For general information about subresources in the Kubernetes API, see Kubernetes API Concepts.

Algorithm details

From the most basic perspective, the HorizontalPodAutoscaler controller operates on the ratio between desired metric value and current metric value:

$$desiredReplicas = ceil\left[currentReplicas \times \frac{currentMetricValue}{desiredMetricValue}\right]$$

For example, if the current metric value is `200m` , and the desired value is `100m` , the number of replicas will be doubled, since $200.0 \div 100.0 = 2.0$.
If the current value is instead `50m` , you'll halve the number of replicas, since $50.0 \div 100.0 = 0.5$. The control plane skips any scaling action if the ratio is sufficiently close to 1.0 (within a configurable tolerance, 0.1 by default).

When a `targetAverageValue` or `targetAverageUtilization` is specified, the `currentMetricValue` is computed by taking the average of the given metric across all Pods in the HorizontalPodAutoscaler's scale target.

Before checking the tolerance and deciding on the final values, the control plane also considers whether any metrics are missing, and how many Pods are `Ready` . For per-pod resource metrics, all Pods with a deletion timestamp set (objects with a deletion timestamp are in the process of being shut down / removed) are ignored, and all failed Pods are discarded. For external and object metrics, the replica count is based on the number of Running and Ready Pods; terminating Pods that are still Ready continue to count toward that total.

If a particular Pod is missing metrics, it is set aside for later; Pods with missing metrics will be used to adjust the final scaling amount.

When scaling on CPU, if any pod has yet to become ready (it's still initializing, or possibly is unhealthy) *or* the most recent metric point for the pod was before it became ready, that pod is set aside as well.

Due to technical constraints, the HorizontalPodAutoscaler controller cannot exactly determine the first time a pod becomes ready when determining whether to set aside certain CPU metrics. Instead, it considers a Pod "not yet ready" if it's unready and transitioned to ready within a short, configurable window of time since it started. This value is configured with the `--horizontal-pod-autoscaler-initial-readiness-delay` command line option, and its default is 30 seconds. Once a pod has become ready, it considers any transition to ready to be the first if it occurred within a longer, configurable time since it started. This value is configured with the `--horizontal-pod-autoscaler-cpu-initialization-period` command line option, and its default is 5 minutes.

The $\frac{currentMetricValue}{desiredMetricValue}$ base scale ratio is then calculated, using the remaining pods not set aside or discarded from above.

If there were any missing metrics, the control plane recomputes the average more conservatively, assuming those pods were consuming 100% of the desired value in case of a scale down, and 0% in case of a scale up. This dampens the magnitude of any potential scale.

Furthermore, if any not-yet-ready pods were present, and the workload would have scaled up without factoring in missing metrics or not-yet-ready pods, the controller conservatively assumes that the not-yet-ready pods are consuming 0% of the desired metric, further dampening the magnitude of a scale up.

After factoring in the not-yet-ready pods and missing metrics, the controller recalculates the usage ratio. If the new ratio reverses the scale direction, or is within the tolerance, the controller doesn't take any scaling action. In other cases, the new ratio is used to decide any change to the number of Pods.

Note that the *original* value for the average utilization is reported back via the HorizontalPodAutoscaler status, without factoring in the not-yet-ready pods or missing metrics, even when the new usage ratio is used.

If multiple metrics are specified in a HorizontalPodAutoscaler, this calculation is done for each metric, and then the largest of the desired replica counts is chosen. If any of these metrics cannot be converted into a desired replica count (e.g. due to an error fetching the metrics from the metrics APIs) and a scale down is suggested by the metrics which can be fetched, scaling is skipped. This means that the HPA is still capable of scaling up if one or more metrics give a `desiredReplicas` greater than the current value.

Finally, right before HPA scales the target, the scale recommendation is recorded. The controller considers all recommendations within a configurable window choosing the highest recommendation from within that window. You can configure this value using the `--horizontal-pod-autoscaler-downscale-stabilization` command line option, which defaults to 5 minutes. This means that scaledowns will occur gradually, smoothing out the impact of rapidly fluctuating metric values.

Pod readiness and autoscaling metrics

The HorizontalPodAutoscaler (HPA) controller includes two command line options that influence how CPU metrics are collected from Pods during startup:

1. `--horizontal-pod-autoscaler-cpu-initialization-period` (default: 5 minutes)

This defines the time window after a Pod starts during which its **CPU usage is ignored** unless: - The Pod is in a `Ready` state **and** - The metric sample was taken entirely during the period it was `Ready`.

This command line option helps **exclude misleading high CPU usage** from initializing Pods (for example: Java apps warming up) in HPA scaling decisions.

1. `--horizontal-pod-autoscaler-initial-readiness-delay` (default: 30 seconds)

This defines a short delay period after a Pod starts during which the HPA controller treats Pods that are currently `Unready` as still initializing, **even if they have previously transitioned to `Ready` briefly**.

It is designed to: - Avoid including Pods that rapidly fluctuate between `Ready` and `Unready` during startup. - Ensure stability in the initial readiness signal before HPA considers their metrics valid.

You can only set these command line options cluster-wide.

Key behaviors for pod readiness

- If a Pod is `Ready` and remains `Ready`, it can be counted as contributing metrics even within the delay.
- If a Pod rapidly toggles between `Ready` and `Unready`, metrics are ignored until it's considered stably `Ready`.

Good practice for pod readiness

- Configure a `startupProbe` that doesn't pass until the high CPU usage has passed, or
- Ensure your `readinessProbe` only reports `Ready` **after** the CPU spike subsides, using `initialDelaySeconds`.

And ideally also set `--horizontal-pod-autoscaler-cpu-initialization-period` to **cover the startup duration**.

API object

The HorizontalPodAutoscaler is an API kind in the Kubernetes `autoscaling` API group. The current stable version can be found in the `autoscaling/v2` API version which includes support for scaling on memory and custom metrics. The new fields introduced in `autoscaling/v2` are preserved as annotations when working with `autoscaling/v1`.

When you create a HorizontalPodAutoscaler API object, make sure the name specified is a valid DNS subdomain name. More details about the API object can be found at HorizontalPodAutoscaler Object.

Stability of workload scale

When managing the scale of a group of replicas using the HorizontalPodAutoscaler, it is possible that the number of replicas keeps fluctuating frequently due to the dynamic nature of the metrics evaluated. This is sometimes referred to as *thrashing*, or *flapping*. It's similar to the concept of *hysteresis* in cybernetics.

Autoscaling during rolling update

Kubernetes lets you perform a rolling update on a Deployment. In that case, the Deployment manages the underlying ReplicaSets for you. When you configure autoscaling for a Deployment, you bind a HorizontalPodAutoscaler to a single Deployment. The HorizontalPodAutoscaler manages the `replicas` field of the Deployment. The deployment controller is responsible for setting the `replicas` of the underlying ReplicaSets so that they add up to a suitable number during the rollout and also afterwards.

If you perform a rolling update of a StatefulSet that has an autoscaled number of replicas, the StatefulSet directly manages its set of Pods (there is no intermediate resource similar to ReplicaSet).

Support for resource metrics

Any HPA target can be scaled based on the resource usage of the pods in the scaling target. When defining the pod specification the resource requests like `cpu` and `memory` should be specified. This is used to determine the resource utilization and used by the HPA controller to scale the target up or down. To use resource utilization based scaling specify a metric source like this:

```yaml
type: Resource
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 60
```

With this metric the HPA controller will keep the average utilization of the pods in the scaling target at 60%. Utilization is the ratio between the current usage of resource to the requested resources of the pod. See Algorithm for more details about how the utilization is calculated and averaged.

> **Note:**
> Since the resource usages of all the containers are summed up the total pod utilization may not accurately represent the individual container resource usage. This could lead to situations where a single container might be running with high usage and the HPA will not scale out because the overall pod usage is still within acceptable limits.

Container resource metrics

ⓘ **FEATURE STATE:** Kubernetes v1.30 [stable](enabled by default)

The HorizontalPodAutoscaler API also supports a container metric source where the HPA can track the resource usage of individual containers across a set of Pods, in order to scale the target resource. This lets you configure scaling thresholds for the containers that matter most in a particular Pod. For example, if you have a web application and a sidecar container that provides logging, you can scale based on the resource use of the web application, ignoring the sidecar container and its resource use.

If you revise the target resource to have a new Pod specification with a different set of containers, you should revise the HPA spec if that newly added container should also be used for scaling. If the specified container in the metric source is not present or only present in a subset of the pods then those pods are ignored and the recommendation is recalculated. See Algorithm for more details about the calculation. To use container resources for autoscaling define a metric source as follows:

```yaml
type: ContainerResource
containerResource:
  name: cpu
  container: application
  target:
    type: Utilization
    averageUtilization: 60
```

In the above example the HPA controller scales the target such that the average utilization of the cpu in the `application` container of all the pods is 60%.

> **Note:**
> If you change the name of a container that a HorizontalPodAutoscaler is tracking, you can make that change in a specific order to ensure scaling remains available and effective whilst the change is being applied. Before you update the resource that defines the container (such as a Deployment), you should update the associated HPA to track both the new and old container names. This way, the HPA is able to calculate a scaling recommendation throughout the update process.
>
> Once you have rolled out the container name change to the workload resource, tidy up by removing the old container name from the HPA specification.

Scaling on custom metrics

ⓘ **FEATURE STATE:** Kubernetes v1.23 [stable]

(the `autoscaling/v2beta2` API version previously provided this ability as a beta feature)

Provided that you use the `autoscaling/v2` API version, you can configure a HorizontalPodAutoscaler to scale based on a custom metric (that is not built in to Kubernetes or any Kubernetes component). The HorizontalPodAutoscaler controller then queries for these custom metrics from the Kubernetes API.

See Support for metrics APIs for the requirements.

Scaling on multiple metrics

ⓘ **FEATURE STATE:** Kubernetes v1.23 [stable]

(the `autoscaling/v2beta2` API version previously provided this ability as a beta feature)

Provided that you use the `autoscaling/v2` API version, you can specify multiple metrics for a HorizontalPodAutoscaler to scale on. Then, the HorizontalPodAutoscaler controller evaluates each metric, and proposes a new scale based on that metric. The HorizontalPodAutoscaler takes the maximum scale recommended for each metric and sets the workload to that size (provided that this isn't larger than the overall maximum that you configured).

Support for metrics APIs

By default, the HorizontalPodAutoscaler controller retrieves metrics from a series of APIs. In order for it to access these APIs, cluster administrators must ensure that:

- The API aggregation layer is enabled.
- The corresponding APIs are registered:
  - For resource metrics, this is the `metrics.k8s.io` API, generally provided by metrics-server. It can be launched as a cluster add-on.
  - For custom metrics, this is the `custom.metrics.k8s.io` API. It's provided by "adapter" API servers provided by metrics solution vendors. Check with your metrics pipeline to see if there is a Kubernetes metrics adapter available.
  - For external metrics, this is the `external.metrics.k8s.io` API. It may be provided by the custom metrics adapters provided above.

For more information on these different metrics paths and how they differ please see the relevant design proposals for the HPA V2, custom.metrics.k8s.io and external.metrics.k8s.io.

For examples of how to use them see the walkthrough for using custom metrics and the walkthrough for using external metrics.

Configurable scaling behavior

ⓘ **FEATURE STATE:** Kubernetes v1.23 [stable]

(the `autoscaling/v2beta2` API version previously provided this ability as a beta feature)

If you use the `v2` HorizontalPodAutoscaler API, you can use the `behavior` field (see the API reference) to configure separate scale-up and scale-down behaviors. You specify these behaviors by setting `scaleUp` and / or `scaleDown` under the `behavior` field.

Scaling policies let you control the rate of change of replicas while scaling. Also two settings can be used to prevent flapping: you can specify a *stabilization window* for smoothing replica counts, and a tolerance to ignore minor metric fluctuations below a specified threshold.

Scaling policies

One or more scaling policies can be specified in the `behavior` section of the spec. When multiple policies are specified the policy which allows the highest amount of change is the policy which is selected by default. The following example shows this behavior while scaling down:

```yaml
behavior:
  scaleDown:
    policies:
    - type: Pods
      value: 4
      periodSeconds: 60
    - type: Percent
      value: 10
      periodSeconds: 60
```

`periodSeconds` indicates the length of time in the past for which the policy must hold true. The maximum value that you can set for `periodSeconds` is 1800 (half an hour). The first policy *(Pods)* allows at most 4 replicas to be scaled down in one minute. The second policy *(Percent)* allows at most 10% of the current replicas to be scaled down in one minute.

Since by default the policy which allows the highest amount of change is selected, the second policy will only be used when the number of pod replicas is more than 40. With 40 or less replicas, the first policy will be applied. For instance if there are 80 replicas and the target has to be scaled down to 10 replicas then during the first step 8 replicas will be reduced. In the next iteration when the number of replicas is 72, 10% of the pods is 7.2 but the number is rounded up to 8. On each loop of the autoscaler controller the number of pods to be change is re-calculated based on the number of current replicas. When the number of replicas falls below 40 the first policy *(Pods)* is applied and 4 replicas will be reduced at a time.

The policy selection can be changed by specifying the `selectPolicy` field for a scaling direction. By setting the value to `Min` which would select the policy which allows the smallest change in the replica count. Setting the value to `Disabled` completely disables scaling in that direction.

Stabilization window

The stabilization window is used to restrict the flapping of replica count when the metrics used for scaling keep fluctuating. The autoscaling algorithm uses this window to infer a previous desired state and avoid unwanted changes to workload scale.

For example, in the following example snippet, a stabilization window is specified for `scaleDown`.

```yaml
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
```

When the metrics indicate that the target should be scaled down the algorithm looks into previously computed desired states, and uses the highest value from the specified interval. In the above example, all desired states from the past 5 minutes will be considered.

This approximates a rolling maximum, and avoids having the scaling algorithm frequently remove Pods only to trigger recreating an equivalent Pod just moments later.

Tolerance

FEATURE STATE: Kubernetes v1.35 [beta](enabled by default)

The `tolerance` field configures a threshold for metric variations, preventing the autoscaler from scaling for changes below that value.

This tolerance is defined as the amount of variation around the desired metric value under which no scaling will occur. For example, consider a HorizontalPodAutoscaler configured with a target memory consumption of 100MiB and a scale-up tolerance of 5%:

```yaml
behavior:
  scaleUp:
    tolerance: 0.05 # 5% tolerance for scale up
```

With this configuration, the HPA algorithm will only consider scaling up if the memory consumption is higher than 105MiB (that is: 5% above the target).

If you don't set this field, the HPA applies the default cluster-wide tolerance of 10%. This default can be updated for both scale-up and scale-down using the kube-controller-manager `--horizontal-pod-autoscaler-tolerance` command line argument. (You can't use the Kubernetes API to configure this default value.)

Default behavior

To use the custom scaling not all fields have to be specified. Only values which need to be customized can be specified. These custom values are merged with default values. The default values match the existing behavior in the HPA algorithm.

```yaml
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
    - type: Pods
      value: 4
      periodSeconds: 15
    selectPolicy: Max
```

For scaling down the stabilization window is *300* seconds (or the value of the `--horizontal-pod-autoscaler-downscale-stabilization` command line option, if provided). There is only a single policy for scaling down which allows a 100% of the currently running replicas to be removed which means the scaling target can be scaled down to the minimum allowed replicas. For scaling up there is no stabilization window. When the metrics indicate that the target should be scaled up the target is scaled up immediately. There are 2 policies where 4 pods or a 100% of the currently running replicas may at most be added every 15 seconds till the HPA reaches its steady state.

Example: change downscale stabilization window

To provide a custom downscale stabilization window of 1 minute, the following behavior would be added to the HPA:

```yaml
behavior:
  scaleDown:
    stabilizationWindowSeconds: 60
```

Example: limit scale down rate

To limit the rate at which pods are removed by the HPA to 10% per minute, the following behavior would be added to the HPA:

```yaml
behavior:
  scaleDown:
    policies:
    - type: Percent
      value: 10
      periodSeconds: 60
```

To ensure that no more than 5 Pods are removed per minute, you can add a second scale-down policy with a fixed size of 5, and set `selectPolicy` to minimum. Setting `selectPolicy` to `Min` means that the autoscaler chooses the policy that affects the smallest number of Pods:

```yaml
behavior:
  scaleDown:
    policies:
    - type: Percent
      value: 10
      periodSeconds: 60
    - type: Pods
      value: 5
      periodSeconds: 60
    selectPolicy: Min
```

Example: disable scale down

The `selectPolicy` value of `Disabled` turns off scaling the given direction. So to prevent downscaling the following policy would be used:

```yaml
behavior:
  scaleDown:
    selectPolicy: Disabled
```

Support for HorizontalPodAutoscaler in kubectl

HorizontalPodAutoscaler, like every API resource, is supported in a standard way by `kubectl`. You can create a new autoscaler using `kubectl create` command. You can list autoscalers by `kubectl get hpa` or get detailed description by `kubectl describe hpa`. Finally, you can delete an autoscaler using `kubectl delete hpa`.

In addition, there is a special `kubectl autoscale` command for creating a HorizontalPodAutoscaler object. For instance, executing `kubectl autoscale rs foo --min=2 --max=5 --cpu=80%` will create an autoscaler for ReplicaSet *foo*, with target CPU utilization set to `80%` and the number of replicas between 2 and 5.

Implicit maintenance-mode deactivation

You can implicitly deactivate the HPA for a target without the need to change the HPA configuration itself. If the target's desired replica count is set to 0, and the HPA's minimum replica count is greater than 0, the HPA stops adjusting the target (and sets the `ScalingActive` Condition on itself to `false`) until you reactivate it by manually adjusting the target's desired replica count or HPA's minimum replica count.

Migrating Deployments and StatefulSets to horizontal autoscaling

When an HPA is enabled, it is recommended that the value of `spec.replicas` of the Deployment and / or StatefulSet be removed from their manifest(s). If this isn't done, any time a change to that object is applied, for example via `kubectl apply -f deployment.yaml`, this will instruct Kubernetes to scale the current number of Pods to the value of the `spec.replicas` key. This may not be desired and could be troublesome when an HPA is active, resulting in thrashing or flapping behavior.

Keep in mind that the removal of `spec.replicas` may incur a one-time degradation of Pod counts as the default value of this key is 1 (reference Deployment Replicas). Upon the update, all Pods except 1 will begin their termination procedures. Any deployment application afterwards will behave as normal and respect a rolling update configuration as desired. You can avoid this degradation by choosing one of the following two methods based on how you are modifying your deployments:

Client Side Apply (this is the default) | Server Side Apply

1. `kubectl apply edit-last-applied deployment/<deployment_name>`
2. In the editor, remove `spec.replicas`. When you save and exit the editor, `kubectl` applies the update. No changes to Pod counts happen at this step.
3. You can now remove `spec.replicas` from the manifest. If you use source code management, also commit your changes or take whatever other steps for revising the source code are appropriate for how you track updates.
4. From here on out you can run `kubectl apply -f deployment.yaml`

What's next

If you configure autoscaling in your cluster, you may also want to consider using node autoscaling to ensure you are running the right number of nodes. You can also read more about *vertical* Pod autoscaling.

For more information on HorizontalPodAutoscaler:

- Read a walkthrough example for horizontal pod autoscaling.
- Read documentation for `kubectl autoscale`.
- If you would like to write your own custom metrics adapter, check out the boilerplate to get started.
- Read the API reference for HorizontalPodAutoscaler.

Feedback

Was this page helpful?

Yes No

Last modified March 15, 2026 at 3:21 PM PST: fix: replace deprecated argument `--cpu-percent` with `--cpu` (af93a0a732)

Vertical Pod Autoscaling

Vertical Pod Autoscaling

In Kubernetes, a *VerticalPodAutoscaler* automatically updates a workload management resource (such as a Deployment or StatefulSet), with the aim of automatically adjusting infrastructure resource requests and limits to match actual usage.

Vertical scaling means that the response to increased resource demand is to assign more resources (for example: memory or CPU) to the Pods that are already running for the workload. This is also known as *rightsizing*, or sometimes *autopilot*. This is different from horizontal scaling, which for Kubernetes would mean deploying more Pods to distribute the load.

If the resource usage decreases, and the Pod resource requests are above optimal levels, the VerticalPodAutoscaler instructs the workload resource (the Deployment, StatefulSet, or other similar resource) to adjust resource requests back down, preventing resource waste.

The VerticalPodAutoscaler is implemented as a Kubernetes API resource and a controller. The resource determines the behavior of the controller. The vertical pod autoscaling controller, running within the Kubernetes data plane, periodically adjusts the resource requests and limits of its target (for example, a Deployment) based on analysis of historical resource utilization, the amount of resources available in the cluster, and real-time events such as out-of-memory (OOM) conditions.

API object

The VerticalPodAutoscaler is defined as a Custom Resource Definition (CRD) in Kubernetes. Unlike HorizontalPodAutoscaler, which is part of the core Kubernetes API, VPA must be installed separately in your cluster.

The current stable API version is `autoscaling.k8s.io/v1`. More details about the VPA installation and API can be found in the VPA GitHub repository.

How does a VerticalPodAutoscaler work?

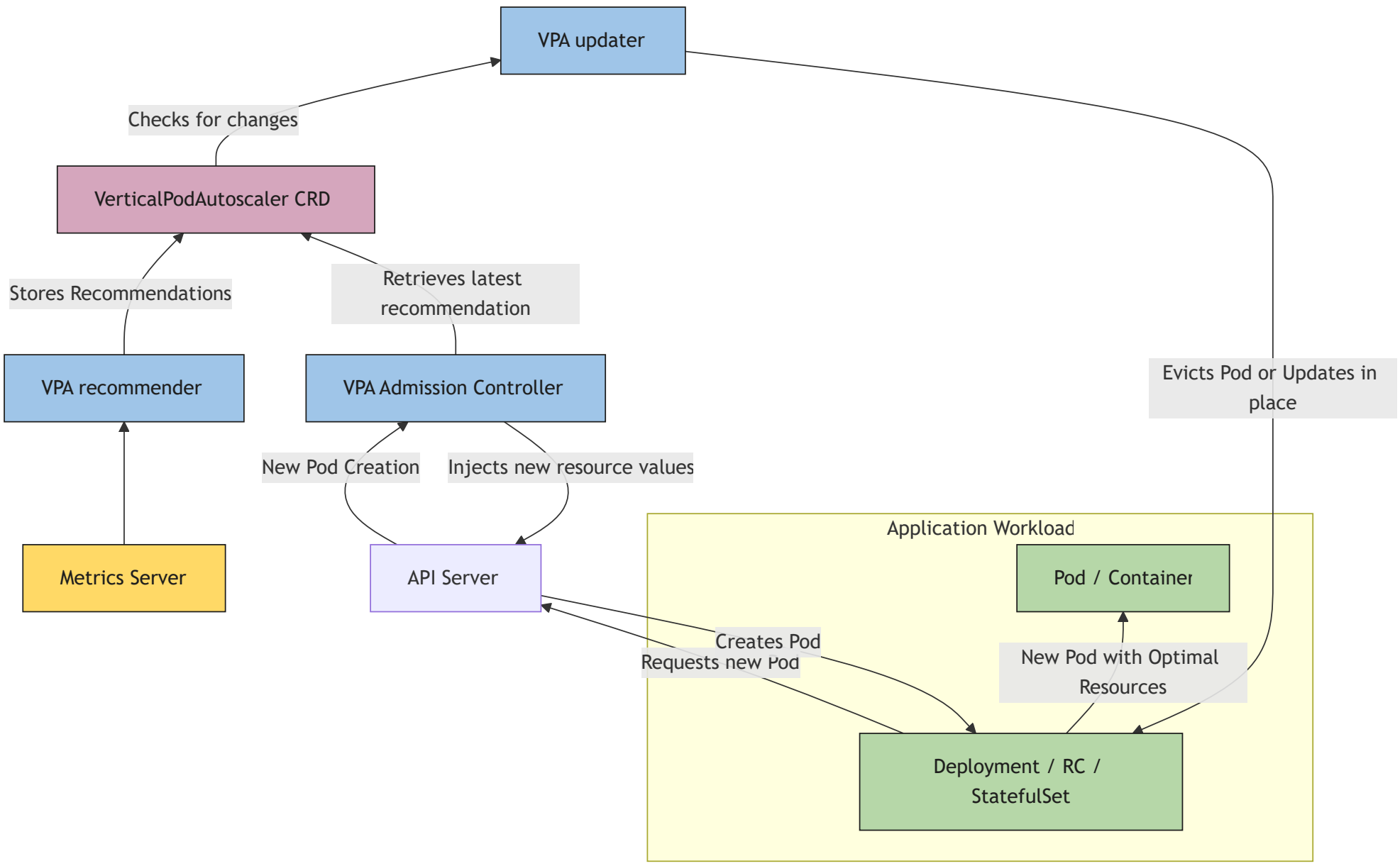


Figure 1. VerticalPodAutoscaler controls the resource requests and limits of Pods in a Deployment

Kubernetes implements vertical pod autoscaling through multiple cooperating components that run intermittently (it is not a continuous process). The VPA consists of three main components:

- The *recommender*, which analyzes resource usage and provides recommendations.
- The *updater*, that Pod resource requests either by evicting Pods or modifying them in place.
- And the VPA *admission controller* webhook, which applies resource recommendations to new or recreated Pods.

Once during each period, the Recommender queries the resource utilization for Pods targeted by each VerticalPodAutoscaler definition. The Recommender finds the target resource defined by the `targetRef`, then selects the pods based on the target resource's `.spec.selector` labels, and obtains the metrics from the resource metrics API to analyze actual CPU and memory consumption.

The Recommender analyzes both current and historical resource usage data (CPU and memory) for each Pod targeted by the VerticalPodAutoscaler. It examines:

- Historical consumption patterns over time to identify trends
- Peak usage and variance to ensure sufficient headroom
- Out-of-memory (OOM) events and other resource-related incidents

Based on this analysis, the Recommender calculates three types of recommendations:

- Target recommendation (optimal resources for typical usage)
- Lower bound (minimum viable resources)
- Upper bound (maximum reasonable resources).

These recommendations are stored in the VerticalPodAutoscaler resource's `.status.recommendation` field.

The *updater* component monitors the VerticalPodAutoscaler resources and compares current Pod resource requests with the recommendations. When the difference exceeds configured thresholds and the update policy allows it, the updater can either:

- Evict Pods, triggering their recreation with new resource requests (traditional approach)
- Update Pod resources in place without eviction, when the cluster supports in-place Pod resource updates

The chosen method depends on the configured update mode, cluster capabilities, and the type of resource change needed. In-place updates, when available, avoid Pod disruption but may have limitations on which resources can be modified. The updater respects PodDisruptionBudgets to minimize service impact.

The *admission controller* operates as a mutating webhook that intercepts Pod creation requests. It checks if the Pod is targeted by a VerticalPodAutoscaler and, if so, applies the recommended resource requests and limits before the Pod is created. More specifically, the admission controller uses the Target recommendation in the VerticalPodAutoscaler resource's `.status.recommendation` stanza as the new resource requests. The admission controller ensures new Pods start with appropriately sized resource allocations, whether they're created during initial deployment, after an eviction by the updater, or due to scaling operations.

The VerticalPodAutoscaler requires a metrics source, such as Kubernetes' Metrics Server add-on, to be installed in the cluster. The VPA components fetch metrics from the `metrics.k8s.io` API. The Metrics Server needs to be launched separately as it is not deployed by default in most clusters. For more information about resource metrics, see Metrics Server.

Update modes

A VerticalPodAutoscaler supports different *update modes* that control how and when resource recommendations are applied to your Pods. You configure the update mode using the `updateMode` field in the VPA spec under `updatePolicy`:

```yaml
---
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Recreate"  # Off, Initial, Recreate, InPlaceOrRecreate
```

Off

In the *Off* update mode, the VPA recommender still analyzes resource usage and generates recommendations, but these recommendations are not automatically applied to Pods. The recommendations are only stored in the VPA object's `.status` field.

You can use a tool such as `kubectl` to view the `.status` and the recommendations in it.

Initial

In *Initial* mode, VPA only sets resource requests when Pods are first created. It does not update resources for already running Pods, even if recommendations change over time. The recommendations apply only during Pod creation.

Recreate

In *Recreate* mode, VPA actively manages Pod resources by evicting Pods when their current resource requests differ significantly from recommendations. When a Pod is evicted, the workload controller (managing a Deployment, StatefulSet, etc) creates a replacement Pod, and the VPA admission controller applies the updated resource requests to the new Pod.

InPlaceOrRecreate

In `InPlaceOrRecreate` mode, VPA attempts to update Pod resource requests and limits without restarting the Pod when possible. However, if in-place updates cannot be performed for a particular resource change, VPA falls back to evicting the Pod (similar to `Recreate` mode) and allowing the workload controller to create a replacement Pod with updated resources.

In this mode, the updater applies recommendations in-place using the Resize Container Resources In-Place feature.

Auto (deprecated)

> **Note:**
> The `Auto` update mode is **deprecated since VPA version 1.4.0**. Use `Recreate` for eviction-based updates, or `InPlaceOrRecreate` for in-place updates with eviction fallback.

`Auto` mode is currently an alias for `Recreate` mode and behaves identically. It was introduced to allow for future expansion of automatic update strategies.

Resource policies

Resource policies allow you to fine-tune how the VerticalPodAutoscaler generates recommendations and applies updates. You can set boundaries for resource recommendations, specify which resources to manage, and configure different policies for individual containers within a Pod.

You define resource policies in the `resourcePolicy` field of the VPA spec:

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: my-app-vpa
spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: my-app
  updatePolicy:
    updateMode: "Recreate"
  resourcePolicy:
    containerPolicies:
    - containerName: "application"
      minAllowed:
        cpu: 100m
        memory: 128Mi
      maxAllowed:
        cpu: 2
        memory: 2Gi
      controlledResources:
      - cpu
      - memory
      controlledValues: RequestsAndLimits
```

minAllowed and maxAllowed

These fields set boundaries for VPA recommendations. The VPA will never recommend resources below `minAllowed` or above `maxAllowed`, even if the actual usage data suggests different values.

controlledResources

The `controlledResources` field specifies which resource types VPA should manage for a container in a Pod. If not specified, VPA manages both CPU and memory by default. You can restrict VPA to manage only specific resources. Valid resource names include `cpu` and `memory`.

controlledValues

The `controlledValues` field determines whether VPA controls resource requests, limits, or both:

RequestsAndLimits

VPA sets both requests and limits. The limit scales proportionally to the request based on the request-to-limit ratio defined in the Pod spec. This is the default mode.

RequestsOnly

VPA only sets requests, leaving limits unchanged. Limits are respected and can still trigger throttling or out-of-memory kills if usage exceeds them.

See requests and limits to learn more about those two concepts.

LimitRange resources

The admission controller and updater VPA components post-process recommendations to comply with the constraints defined in LimitRanges. The LimitRange resources with `type` Pod and Container are checked in the Kubernetes cluster.

For example, if the `max` field in a Container LimitRange resource is exceeded, both VPA components lower the limit to the value defined in the `max` field, and the request is proportionally decreased to maintain the request-to-limit ratio in the Pod spec.

What's next

If you configure autoscaling in your cluster, you may also want to consider using node autoscaling to ensure you are running the right number of nodes. You can also read more about *horizontal* Pod autoscaling.

Feedback

Was this page helpful?

Yes No

Last modified January 28, 2026 at 10:39 AM PST: Add figure caption and reference Mermaid source in diagram (5be933e662)

Node Shutdowns

Node Shutdowns

In a Kubernetes cluster, a node can be shut down in a planned graceful way or unexpectedly because of reasons such as a power outage or something else external. A node shutdown could lead to workload failure if the node is not drained before the shutdown. A node shutdown can be either **graceful** or **non-graceful**.

> **Caution:**
>
> The `unattended-upgrades` package from Debian conflicts with node graceful shutdown in its normal configuration. If you use the default configuration of `unattended-upgrades`, which customizes the server shutdown grace period, then the kubelet fails to obtain the necessary lock to handle shutdown events properly.
>
> This happens if the `shutdownGracePeriod` value is greater than 30 seconds. To avoid this, you can suppress part of the `unattended-upgrades` configuration, by making `/etc/systemd/logind.conf.d/unattended-upgrades-logind-maxdelay.conf` be a symbolic link to `/dev/null`.
>
> For more details, refer to the `logind.conf` documentation.

Graceful node shutdown

The kubelet attempts to detect node system shutdown and terminates pods running on the node.

Kubelet ensures that pods follow the normal pod termination process during the node shutdown. During node shutdown, the kubelet does not accept new Pods (even if those Pods are already bound to the node).

Enabling graceful node shutdown

Linux | Windows

FEATURE STATE: Kubernetes v1.21 [beta](enabled by default)

On Linux, the graceful node shutdown feature is controlled with the `GracefulNodeShutdown` feature gate which is enabled by default in 1.21.

> **Note:**
> The graceful node shutdown feature depends on systemd since it takes advantage of systemd inhibitor locks to delay the node shutdown with a given duration.

Configuring graceful node shutdown

Note that by default, both configuration options described below, `shutdownGracePeriod` and `shutdownGracePeriodCriticalPods`, are set to zero, thus not activating the graceful node shutdown functionality. To activate the feature, both options should be configured appropriately and set to non-zero values.

Once the kubelet is notified of a node shutdown, it sets a `NotReady` condition on the Node, with the `reason` set to `"node is shutting down"`. The kube-scheduler honors this condition and does not schedule any Pods onto the affected node; other third-party schedulers are expected to follow the same logic. This means that new Pods won't be scheduled onto that node and therefore none will start.

The kubelet **also** rejects Pods during the `PodAdmission` phase if an ongoing node shutdown has been detected, so that even Pods with a toleration for `node.kubernetes.io/not-ready:NoSchedule` do not start there.

When kubelet is setting that condition on its Node via the API, the kubelet also begins terminating any Pods that are running locally.

During a graceful shutdown, kubelet terminates pods in two phases:

1. Terminate regular pods running on the node.
2. Terminate critical pods running on the node.

The graceful node shutdown feature is configured with two `KubeletConfiguration` options:

- `shutdownGracePeriod`：

  Specifies the total duration that the node should delay the shutdown by. This is the total grace period for pod termination for both regular and critical pods.

- `shutdownGracePeriodCriticalPods`：

  Specifies the duration used to terminate critical pods during a node shutdown. This value should be less than `shutdownGracePeriod`.

> **Note:**
> There are cases when Node termination was cancelled by the system (or perhaps manually by an administrator). In either of those situations the Node will return to the `Ready` state. However, Pods which already started the process of termination will not be restored by kubelet and will need to be re-scheduled.

For example, if `shutdownGracePeriod=30s`, and `shutdownGracePeriodCriticalPods=10s`, kubelet will delay the node shutdown by 30 seconds. During the shutdown, the first 20 (30-10) seconds would be reserved for gracefully terminating normal pods, and the last 10 seconds would be reserved for terminating critical pods.

> **Note:**
> When pods were evicted during the graceful node shutdown, they are marked as shutdown. Running `kubectl get pods` shows the status of the evicted pods as `Terminated`. And `kubectl describe pod` indicates that the pod was evicted because of node shutdown:
>
> ```
> Reason:         Terminated
> Message:        Pod was terminated in response to imminent node shutdown.
> ```

Pod Priority based graceful node shutdown

FEATURE STATE: Kubernetes v1.24 [beta](enabled by default)

To provide more flexibility during graceful node shutdown around the ordering of pods during shutdown, graceful node shutdown honors the PriorityClass for Pods, provided that you enabled this feature in your cluster. The feature allows cluster administrators to explicitly define the ordering of pods during graceful node shutdown based on priority classes.

The Graceful Node Shutdown feature, as described above, shuts down pods in two phases, non-critical pods, followed by critical pods. If additional flexibility is needed to explicitly define the ordering of pods during shutdown in a more granular way, pod priority based graceful shutdown can be used.

When graceful node shutdown honors pod priorities, this makes it possible to do graceful node shutdown in multiple phases, each phase shutting down a particular priority class of pods. The kubelet can be configured with the exact phases and shutdown time per phase.

Assuming the following custom pod priority classes in a cluster,

Pod priority class name	Pod priority class value
`custom-class-a`	100000

Pod priority class name	Pod priority class value
`custom-class-b`	10000
`custom-class-c`	1000
`regular/unset`	0

Within the kubelet configuration the settings for `shutdownGracePeriodByPodPriority` could look like:

Pod priority class value	Shutdown period
100000	10 seconds
10000	180 seconds
1000	120 seconds
0	60 seconds

The corresponding kubelet config YAML configuration would be:

```yaml
shutdownGracePeriodByPodPriority:
  - priority: 100000
    shutdownGracePeriodSeconds: 10
  - priority: 10000
    shutdownGracePeriodSeconds: 180
  - priority: 1000
    shutdownGracePeriodSeconds: 120
  - priority: 0
    shutdownGracePeriodSeconds: 60
```

The above table implies that any pod with `priority` value >= 100000 will get just 10 seconds to shut down, any pod with value >= 10000 and < 100000 will get 180 seconds to shut down, any pod with value >= 1000 and < 10000 will get 120 seconds to shut down. Finally, all other pods will get 60 seconds to shut down.

One doesn't have to specify values corresponding to all of the classes. For example, you could instead use these settings:

Pod priority class value	Shutdown period
100000	300 seconds
1000	120 seconds
0	60 seconds

In the above case, the pods with `custom-class-b` will go into the same bucket as `custom-class-c` for shutdown.

If there are no pods in a particular range, then the kubelet does not wait for pods in that priority range. Instead, the kubelet immediately skips to the next priority class value range.

If this feature is enabled and no configuration is provided, then no ordering action will be taken.

Using this feature requires enabling the `GracefulNodeShutdownBasedOnPodPriority` feature gate, and setting `ShutdownGracePeriodByPodPriority` in the kubelet config to the desired configuration containing the pod priority class values and their respective shutdown periods.

> **Note:**

> The ability to take Pod priority into account during graceful node shutdown was introduced as an Alpha feature in Kubernetes v1.23. In Kubernetes 1.36 the feature is Beta and is enabled by default.

Metrics `graceful_shutdown_start_time_seconds` and `graceful_shutdown_end_time_seconds` are emitted under the kubelet subsystem to monitor node shutdowns.

Non-graceful node shutdown handling

ⓘ **FEATURE STATE:** Kubernetes v1.28 [stable](enabled by default)

A node shutdown action may not be detected by kubelet's Node Shutdown Manager, either because the command does not trigger the inhibitor locks mechanism used by kubelet or because of a user error, i.e., the ShutdownGracePeriod and ShutdownGracePeriodCriticalPods are not configured properly. Please refer to above section Graceful Node Shutdown for more details.

When a node is shutdown but not detected by kubelet's Node Shutdown Manager, the pods that are part of a StatefulSet will be stuck in terminating status on the shutdown node and cannot move to a new running node. This is because kubelet on the shutdown node is not available to delete the pods so the StatefulSet cannot create a new pod with the same name. If there are volumes used by the pods, the VolumeAttachments will not be deleted from the original shutdown node so the volumes used by these pods cannot be attached to a new running node. As a result, the application running on the StatefulSet cannot function properly. If the original shutdown node comes up, the pods will be deleted by kubelet and new pods will be created on a different running node. If the original shutdown node does not come up, these pods will be stuck in terminating status on the shutdown node forever.

To mitigate the above situation, a user can manually add the taint `node.kubernetes.io/out-of-service` with either `NoExecute` or `NoSchedule` effect to a Node marking it out-of-service. If a Node is marked out-of-service with this taint, the pods on the node will be forcefully deleted if there are no matching tolerations on it and volume detach operations for the pods terminating on the node will happen immediately. This allows the Pods on the out-of-service node to recover quickly on a different node.

During a non-graceful shutdown, Pods are terminated in the two phases:

1. Force delete the Pods that do not have matching `out-of-service` tolerations.
2. Immediately perform detach volume operation for such pods.

> **Note:**
> - Before adding the taint `node.kubernetes.io/out-of-service` , it should be verified that the node is already in shutdown or power off state (not in the middle of restarting).
> - The user is required to manually remove the out-of-service taint after the pods are moved to a new node and the user has checked that the shutdown node has been recovered since the user was the one who originally added the taint.

Forced storage detach on timeout

In any situation where a pod deletion has not succeeded for 6 minutes, kubernetes will force detach volumes being unmounted if the node is unhealthy at that instant. Any workload still running on the node that uses a force-detached volume will cause a violation of the CSI specification, which states that `ControllerUnpublishVolume` "**must** be called after all `NodeUnstageVolume` and `NodeUnpublishVolume` on the volume are called and succeed". In such circumstances, volumes on the node in question might encounter data corruption.

The forced storage detach behaviour is optional; users might opt to use the "Non-graceful node shutdown" feature instead.

Force storage detach on timeout can be disabled by setting the `disable-force-detach-on-timeout` config field in `kube-controller-manager` . Disabling the force detach on timeout feature means that a volume that is hosted on a node that is unhealthy for more than 6 minutes will not have its associated VolumeAttachment deleted.

After this setting has been applied, unhealthy pods still attached to volumes must be recovered via the Non-Graceful Node Shutdown procedure mentioned above.

> **Note:**
> - Caution must be taken while using the Non-Graceful Node Shutdown procedure.
> - Deviation from the steps documented above can result in data corruption.

What's next

Learn more about the following:

- Blog: Non-Graceful Node Shutdown.
- Cluster Architecture: Nodes.

Feedback

Was this page helpful?

Yes No

Last modified March 13, 2026 at 5:55 PM PST: Add unattended-upgrades conflict warning to node shutdown docs (ec61f9f3e4)

Cluster Networking

Cluster Networking

Networking is a central part of Kubernetes, but it can be challenging to understand exactly how it is expected to work. There are 4 distinct networking problems to address:

1. Highly-coupled container-to-container communications: this is solved by Pods and `localhost` communications.
2. Pod-to-Pod communications: this is the primary focus of this document.
3. Pod-to-Service communications: this is covered by Services.
4. External-to-Service communications: this is also covered by Services.

Kubernetes is all about sharing machines among applications. Typically, sharing machines requires ensuring that two applications do not try to use the same ports. Coordinating ports across multiple developers is very difficult to do at scale and exposes users to cluster-level issues outside of their control.

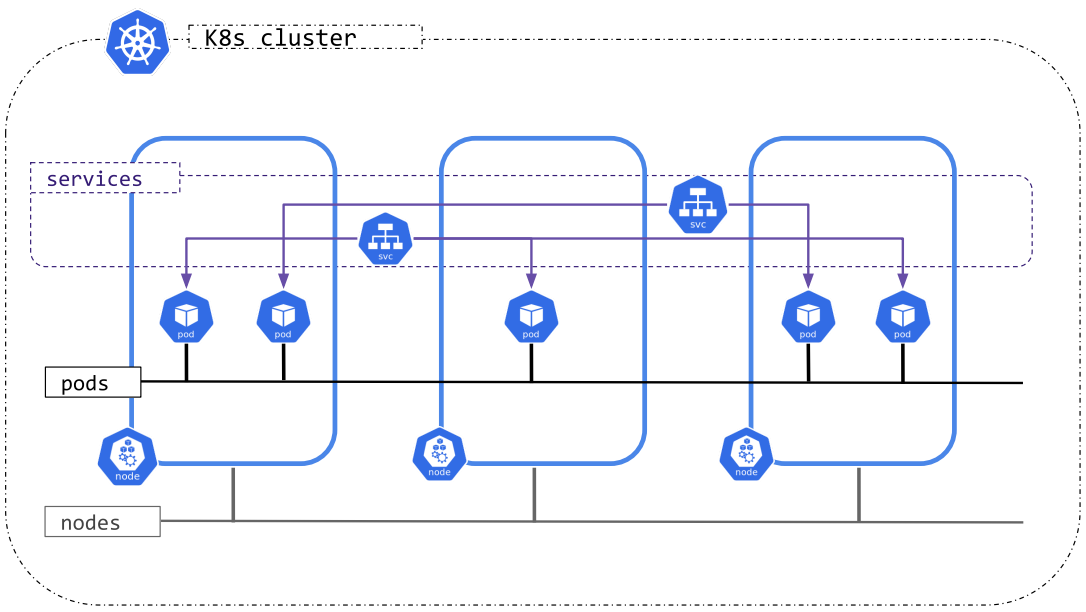
Dynamic port allocation brings a lot of complications to the system - every application has to take ports as flags, the API servers have to know how to insert dynamic port numbers into configuration blocks, services have to know how to find each other, etc. Rather than deal with this, Kubernetes takes a different approach.

To learn about the Kubernetes networking model, see here.

Kubernetes IP address ranges

Kubernetes clusters require to allocate non-overlapping IP addresses for Pods, Services and Nodes, from a range of available addresses configured in the following components:

- The network plugin is configured to assign IP addresses to Pods.
- The kube-apiserver is configured to assign IP addresses to Services.
- The kubelet or the cloud-controller-manager is configured to assign IP addresses to Nodes.



Cluster networking types

Kubernetes clusters, attending to the IP families configured, can be categorized into:

- IPv4 only: The network plugin, kube-apiserver and kubelet/cloud-controller-manager are configured to assign only IPv4 addresses.
- IPv6 only: The network plugin, kube-apiserver and kubelet/cloud-controller-manager are configured to assign only IPv6 addresses.
- IPv4/IPv6 or IPv6/IPv4 dual-stack:
  - The network plugin is configured to assign IPv4 and IPv6 addresses.
  - The kube-apiserver is configured to assign IPv4 and IPv6 addresses.

- The kubelet or cloud-controller-manager is configured to assign IPv4 and IPv6 address.
- All components must agree on the configured primary IP family.

Kubernetes clusters only consider the IP families present on the Pods, Services and Nodes objects, independently of the existing IPs of the represented objects. For example, a server or a pod can have multiple IP addresses assigned to its interfaces, but only the IP addresses in `node.status.addresses` or `pod.status.ips` are considered when implementing the Kubernetes network model and defining the cluster type.

How to implement the Kubernetes network model

The network model is implemented by the container runtime on each node. The most common container runtimes use Container Network Interface (CNI) plugins to manage their network and security capabilities. Many different CNI plugins exist from many different vendors. Some of these provide only basic features of adding and removing network interfaces, while others provide more sophisticated solutions, such as integration with other container orchestration systems, running multiple CNI plugins, advanced IPAM features etc.

See this page for a non-exhaustive list of networking addons supported by Kubernetes.

What's next

The early design of the networking model and its rationale are described in more detail in the networking design document. For future plans and some on-going efforts that aim to improve Kubernetes networking, please refer to the SIG-Network KEPs.

Feedback

Was this page helpful?

Yes No

Last modified October 26, 2025 at 2:21 PM PST: Update content/en/docs/concepts/cluster-administration/networking.md (be90770c29)

Cluster Administration

Cluster Administration

Lower-level detail relevant to creating or administering a Kubernetes cluster.

The cluster administration overview is for anyone creating or administering a Kubernetes cluster. It assumes some familiarity with core Kubernetes concepts.

Planning a cluster

See the guides in Setup for examples of how to plan, set up, and configure Kubernetes clusters. The solutions listed in this article are called *distros*.

> **Note:**
> Not all distros are actively maintained. Choose distros which have been tested with a recent version of Kubernetes.

Before choosing a guide, here are some considerations:

- Do you want to try out Kubernetes on your computer, or do you want to build a high-availability, multi-node cluster? Choose distros best suited for your needs.
- Will you be using **a hosted Kubernetes cluster**, such as Google Kubernetes Engine, or **hosting your own cluster**?
- Will your cluster be **on-premises**, or **in the cloud (IaaS)**? Kubernetes does not directly support hybrid clusters. Instead, you can set up multiple clusters.
- **If you are configuring Kubernetes on-premises**, consider which networking model fits best.
- Will you be running Kubernetes on **"bare metal" hardware** or on **virtual machines (VMs)**?
- Do you **want to run a cluster**, or do you expect to do **active development of Kubernetes project code**? If the latter, choose an actively-developed distro. Some distros only use binary releases, but offer a greater variety of choices.
- Familiarize yourself with the components needed to run a cluster.

Managing a cluster

- Learn how to manage nodes.
  - Read about Node autoscaling.
- Learn how to set up and manage the resource quota for shared clusters.

Securing a cluster

- Generate Certificates describes the steps to generate certificates using different tool chains.
- Kubernetes Container Environment describes the environment for Kubelet managed containers on a Kubernetes node.
- Controlling Access to the Kubernetes API describes how Kubernetes implements access control for its own API.
- Authenticating explains authentication in Kubernetes, including the various authentication options.
- Authorization is separate from authentication, and controls how HTTP calls are handled.
- Using Admission Controllers explains plug-ins which intercepts requests to the Kubernetes API server after authentication and authorization.

- Admission Webhook Good Practices provides good practices and considerations when designing mutating admission webhooks and validating admission webhooks.
- Using Sysctls in a Kubernetes Cluster describes to an administrator how to use the `sysctl` command-line tool to set kernel parameters .
- Auditing describes how to interact with Kubernetes' audit logs.

Securing the kubelet

- Control Plane-Node communication
- TLS bootstrapping
- Kubelet authentication/authorization

Optional Cluster Services

- DNS Integration describes how to resolve a DNS name directly to a Kubernetes service.
- Logging and Monitoring Cluster Activity explains how logging in Kubernetes works and how to implement it.

Feedback

Was this page helpful?

Yes No

Last modified February 03, 2025 at 5:28 PM PST: Add a new page for mutating webhook good practices. (bf971d28d3)